PRACTICAL IMPLEMENTATION MANUAL

TaskFlow Security Automation Lab

Local n8n in Docker + Mockoon

Install the tools. Build the mock website.
Run eight security workflows and verify the results.

Edition Local learning edition | 9 October 2026

Audience Mac users new to Docker and Mockoon; familiar with n8n

Companion TaskFlow_Local_Docker_Mockoon_Starter_Kit.zip

A complete build guide with installation steps, exact commands, mock data, copyable workflow scripts, expected results and recovery checks.

Use invented records only. This is a learning environment, not a production scanner or a real key-rotation service.

Start here

Your first milestone is small: open n8n in Safari, open the TaskFlow website, and make one fake certificate change survive a second API request. Build the eight security workflows only after that works.

This manual uses one main arrangement. Docker runs **two containers**: n8n and the Mockoon server. Mockoon Desktop is the visual editor for the mock API configuration. You will also use Desktop briefly to learn its controls. Do not run the same TaskFlow API in Desktop and Docker at the same time.

What you open	What it is for
Docker Desktop	Start/stop containers and inspect their ports and logs.
Mockoon Desktop	Create or edit mock routes and their starting data.
Safari: http://localhost:5678	Build and run workflows in your local n8n instance.
Safari: http://localhost:3000	View and edit the invented TaskFlow records.
Terminal	Enter the Docker commands printed in this guide.

Follow this order

First session: Chapters 2-5 install Docker and start n8n. Stop when Safari shows your local n8n editor.

Second session: Chapters 6-9 teach Mockoon, start the main mock API, and explain every website control. Stop when an edited certificate is successfully read back.

Then: Chapter 10 establishes the n8n request conventions and reusable task handler. Chapters 11-18 contain the eight workflows. Chapters 19-20 cover reliability, daily use, backups and updates.

Do not mix starter kits. Extract the ZIP named on this edition's cover. The earlier Supabase, MockAPI.io and cloud kits have different addresses, files and record formats.

What is free, and what is not being promised

The lab uses the self-hosted n8n Community edition, which can be used without buying an n8n Cloud subscription. Mockoon's local Desktop and CLI tools are the tools used here, not its paid hosted services. Docker Desktop is free for personal use and education and for qualifying small businesses; organizational use has separate eligibility conditions. Check those conditions before using an employer-managed installation. Hardware, electricity, internet access and your time still have costs. [R1, R2, R3]

This is not an always-on cloud service. Scheduled workflows cannot run normally while your Mac is asleep, shut down, or Docker is stopped. A restart policy is not a substitute for an awake computer.

How to read the examples

A **command** belongs in Terminal. A **JSON body** belongs in the specific editor named in the step. A **Code-node script** belongs in n8n's Code node. Never paste a shell command into Safari or an n8n JavaScript field.

Names such as `cert-001` identify invented records. Text such as `YOUR_LAB_TOKEN` is a placeholder to replace, not a value that is already configured. When a step says **Checkpoint**, verify that result before continuing. Long scripts may wrap or continue onto another page. Copy the complete matching `.js` file from the ZIP to avoid missing a line.

Contents

Begin with Chapters 2-5 for Docker and local n8n. Chapters 6-9 cover Mockoon and the website.

Part I - Automation strategy 4

1. What the TaskFlow lab is building 4

Part II - Install and understand the local environment 6

2. The Docker ideas you actually need 6
3. Install Docker Desktop on your Mac 7
4. Prepare the project folder and Compose file 8
5. Start n8n and open it in Safari 10
6. Install Mockoon Desktop and learn the screen 12
7. Make one small Mockoon API yourself 13
8. Open and run the complete TaskFlow API 14
9. Set up and use the mock website in detail 16
10. Connect n8n and build one reusable task handler 19

Part III - Build the eight workflows 23

11. Workflow 1 - Crypto Asset Discovery 23
12. Workflow 2 - Certificate Expiry Automation 24
13. Workflow 3 - JWT Key Rotation Simulator 26
14. Workflow 4 - PQC Readiness Scanner 28
15. Workflow 5 - Vendor PQC Monitoring 30
16. Workflow 6 - Secret Rotation Simulator 31
17. Workflow 7 - GitHub / CI Crypto Gate 33
18. Workflow 8 - Crypto Incident Response 35

Part IV - Reliability and everyday operation 38

19. Make the lab reliable without overstating it 38
20. Daily use, backup, restore and updates 39

Appendices 43

Appendix A - Mock payloads and endpoint translation 43

Appendix B - Terminal tests 45

Appendix C - Troubleshooting matrix 45

Appendix D - Glossary 47

Appendix E - Completion and preservation checklist 48

References and implementation notes 49

Part I - Automation strategy

1. What the TaskFlow lab is building

The original guide describes a Crypto Control Plane: a connected set of automations that inventories cryptography, evaluates conditions, creates work, tracks remediation and verifies results. This edition keeps that concept and the original eight-workflow sequence. [S1]

1.1 Highest-value opportunities

Opportunity	Purpose	Planning priority
Crypto Asset Discovery	Inventory certificates, signing keys, SSH keys and algorithm use.	Critical
Certificate Expiry Automation	Identify approaching expiry and create renewal work.	Critical
Crypto Deprecation / PQC Review Gate	Flag policy issues or migration-assessment needs before deployment.	Critical
JWT Key Rotation	Detect old signing keys and coordinate a controlled replacement.	Critical
PQC Readiness Dashboard	Track evidence from unknown through verified migration.	Critical
Vendor PQC Tracker	Track vendor commitments, dependencies and review dates.	Critical
Secret Rotation	Review overdue credential metadata without exposing the credential.	Critical
SSH Key Lifecycle	Identify age, ownership and access-review needs.	High
RDS TLS Validation	Check connection encryption and certificate-validation evidence.	High
Crypto Incident Response	Coordinate urgent remediation, verification and closure.	High

These are the source's planning priorities, not automatic severity labels for every detected asset. The eight exercises implement the central ideas; Chapter 19 includes small SSH and RDS extensions.

1.2 The control plane



The loop is **Detect -> Assess -> Create task -> Assign -> Remediate -> Verify -> Close**. A task is not complete merely because a POST request returned success.

1.3 Core rules retained from the first guide

Certificates enter an early-warning tier below 30 days, HIGH below 14 days and CRITICAL below 7 days. Already-expired certificates are CRITICAL. This edition calls the early-warning task priority MEDIUM so that every workflow uses one consistent priority vocabulary.

Signing keys and secrets older than their stated policy produce review or rotation work. The 90-day examples are training policies, not universal requirements. Approvals and checks come before state changes that would be destructive in a real system.

A code review finding identifies a cryptographic pattern requiring investigation. It must not equate the presence of RSA, ECDSA, ECDHE or Ed25519 with evidence of a present compromise. PQC migration, algorithm deprecation, certificate expiry and incident response are distinct questions. [S1, R18]

1.4 PQC readiness: stage, not a safety percentage

Stage	Meaning	Evidence expected
0	Unknown	Inventory or scope is missing.
1	Inventoried	The cryptographic use and owner are identified.
2	Roadmap / migration decision known	A vendor roadmap or internal decision is recorded.
3	Test environment ready	A non-production test environment is documented.
4	Hybrid/PQC tested	Compatibility, performance and rollback evidence exists.
5	Production migrated and verified	Deployment and technical verification are complete.

A later-stage reference does not compensate for missing earlier evidence. Each service record has a specific scope. Testing TLS key establishment does not establish that JWT signatures have migrated. The provided records are simulated and are capped at stage 4; they never prove a real production migration.

1.5 Example of actionable work

```
Title: Review RDS certificate renewal
Priority: HIGH
Owner: Database
Reason: Expiry falls inside the lab warning window
Dependency: Check the application's database connection pool
Rollback: Recorded before any real change
Closure: A fresh observation confirms the new certificate
```

Part II - Install and understand the local environment

2. The Docker ideas you actually need

You do not need to learn all of Docker before using this lab. You do need to understand the following differences so that restarting, changing ports and backing up data do not become guesswork.

Term	Meaning in this lab
Docker Desktop	The Mac application that runs Docker's container engine.
Image	A packaged application, such as `n8nio/n8n:latest`.
Container	A running or stopped instance created from an image.
Volume	Docker-managed storage that remains when its container is replaced.
Bind mount	A connection from a Mac folder to a folder inside a container.
Port	A numbered entry point for a service; n8n uses 5678 here.
Compose	A tool that reads `compose.yaml` and creates the services consistently.
CLI	Command-line interface. Mockoon CLI runs the mock server without a desktop window.

2.1 Why your earlier container did not open in Safari

A Docker PORTS entry showing only `5678/tcp` means the container exposes that port internally but does not show a host-side publication. In this guide, you must see a mapping like:

```
127.0.0.1:5678->5678/tcp
```

The left side is on your Mac. The right side is inside the n8n container. The supplied Compose file creates that connection. A running status alone does not establish that Safari can reach the application. [R4]

2.2 Three addresses, three different jobs

Address	Use it from	Purpose
`http://localhost:5678`	Safari on the Mac	Open the n8n editor.
`http://localhost:3000`	Safari on the Mac	Open the mock website or API.
`http://mock-api:3000`	An HTTP Request node in n8n	Call the other container on the Compose network.

Inside a container, `localhost` refers to that container, not your Mac or another container. Compose registers service names, so n8n can use `mock-api` without you finding a container IP address. Do not copy `http://localhost:3000` from Safari into the lab's n8n HTTP nodes. [R5]

`host.docker.internal` is relevant when a container must call a server running directly on the Mac. It is not the main connection path in this edition. Keeping both lab servers on the same Compose network avoids that additional networking case.

2.3 What is and is not stored permanently

The named volume `taskflow_local_n8n_data` stores n8n's local database, settings and generated encryption configuration. Preserve it and back it up. It is separate from your earlier `n8n_data` volume, if one exists.

Mockoon data buckets remember changes **between requests while the mock server runs**. They do not save those live changes back into the environment file. Stopping/restarting the mock server resets its buckets. The website's snapshot export/restore handles practice-state preservation; it is not an n8n backup. [R6, R7]

> Do not delete containers or volumes to troubleshoot a connection problem. Start with status, port publication and logs. Never run `docker compose down -v` or a volume-pruning command unless you deliberately intend to remove data.

3. Install Docker Desktop on your Mac

Step 1 - Check your Mac before downloading

Click the Apple menu, then **About This Mac**. Record the macOS version and whether the computer shows an Apple chip, such as an M-series chip, or an Intel processor. Choose the corresponding Docker installer. Docker's current requirements include a supported macOS release and at least 4 GB of RAM; the supported macOS window moves over time, so check the installation page rather than assuming an old Mac version is supported. [R2]

Step 2 - Download the official installer

Open the official Docker Mac installation page: https://docs.docker.com/desktop/setup/install/mac-install/. Select **Docker Desktop for Mac with Apple silicon** or **Docker Desktop for Mac with Intel chip** to match Step 1. Download the `.dmg` file. You do not need a cloud server, an n8n Cloud subscription, or a paid Docker plan merely to follow this personal practice setup when you meet Docker's eligibility conditions.

Step 3 - Install and launch

Open the downloaded `Docker.dmg`. Drag Docker into Applications. Then open **Finder -> Applications -> Docker**. Review the license screen and complete the installation prompts. Some releases show recommended/advanced setup choices; others apply defaults and expose those choices in Settings. Do not change networking options for this lab. [R2]

Wait until Docker reports that the engine is running. Merely seeing the Docker window is not enough if it still says starting. A managed computer may require administrator approval; follow the organization's policy rather than bypassing it.

Step 4 - Open Terminal

Press **Command + Space**, type **Terminal**, then press Return. Terminal is where you paste the commands in this manual. Paste one complete block at a time, then press Return. Lines starting with `docker` are commands; output such as `Docker version ...` is not something you type.

Run:

```
docker --version
docker compose version
docker info
```

The first two should print version information. The third should show engine/server information, not a connection error. `docker --version` alone only proves the command is installed; it does not prove the engine is running.

Checkpoint: Docker Desktop says the engine is running, and `docker info` completes. A cannot-connect-to-daemon error means you must finish opening Docker before continuing. A command-not-found error means the CLI installation/PATH needs attention; reopen Terminal and check Docker Settings rather than reinstalling n8n.

Step 5 - Check existing n8n containers without deleting them

Run:

```
docker ps -a
```

Look for the earlier container named `n8n`, `strange_chatterjee`, or another name shown in Docker Desktop. Names are identifiers, not diagnoses. In Docker Desktop, select that old n8n container and click **Stop** before starting this guide's instance. Stop only the n8n container occupying port 5678, not unrelated applications.

Stopping does not delete it. This edition creates a separate lab instance and volume, so your old setup remains available. An empty new editor later is expected; the two instances do not automatically share data. Chapter 20 explains exports and backups.

4. Prepare the project folder and Compose file

Step 1 - Extract the matching ZIP

Download `TaskFlow_Local_Docker_Mockoon_Starter_Kit.zip`. In Finder, double-click it to extract it. Open the extracted `TaskFlow-Local-Lab` folder. Do not open files inside the compressed ZIP and expect Docker to see them.

Move the complete folder to **Documents**. Keep the exact folder name for the commands below. The important layout is:

```
Documents/
  TaskFlow-Local-Lab/
    compose.yaml
    README.md
    mockoon/
      taskflow.mockoon.json
      index.html
      styles.css
      app.js
      seed-data.json
      ci-risky.json
      ci-clean.json
      incident.json
    code/
    events/
    examples/
    practice/
    backups/
    workflow-exports/
```

Step 2 - Move Terminal into that folder

Paste:

```
cd "$HOME/Documents/TaskFlow-Local-Lab"
pwd
ls
```

`cd` changes Terminal's working folder. `$HOME` expands to your Mac home folder. `pwd` prints where you are. `ls` lists the files there. None of these commands deletes anything.

Checkpoint: `ls` includes `compose.yaml` and `mockoon`. If not, locate the extracted folder in Finder. Type `cd ` with a space, drag that folder from Finder into Terminal, and press Return. Then run `pwd` and `ls` again.

Step 3 - Understand the file before running it

The ZIP already contains `compose.yaml`. Open it in a plain-text editor only to inspect it; you do not need to retype it. A YAML file uses indentation to show which settings belong to each service. Do not replace spaces with tabs or save the file as `compose.yaml.txt`.

Copyable file: `compose.yaml`

```
name: taskflow-local
services:
  n8n:
    image: n8nio/n8n:latest
    restart: unless-stopped
    ports:
      - "127.0.0.1:5678:5678"
    environment:
      TZ: Asia/Manila
      GENERIC_TIMEZONE: Asia/Manila
      N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS: "true"
      N8N_RUNNERS_ENABLED: "true"
      N8N_RUNNERS_MODE: internal
      N8N_SECURE_COOKIE: "false"
      N8N_DIAGNOSTICS_ENABLED: "false"
    volumes:
      - n8n_data:/home/node/.n8n
  mock-api:
    image: mockoon/cli:latest
    restart: unless-stopped
    command:
      - --data
      - /data/taskflow.mockoon.json
      - --port
      - "3000"
      - --hostname
      - 0.0.0.0
    ports:
      - "127.0.0.1:3000:3000"
    volumes:
      - ./mockoon:/data:ro
volumes:
  n8n_data:
    name: taskflow_local_n8n_data
```

Step 4 - Read the important settings

Setting	Why it is here
`name: taskflow-local`	Keeps the Compose project's names consistent when the folder is moved.
n8n `image`	Downloads the packaged n8n application; no manual Node.js install is needed.
`127.0.0.1:5678:5678`	Publishes n8n to the Mac's loopback interface.
`n8n_data:/home/node/.n8n`	Persists the application's data outside its replaceable container.
`TZ` and `GENERIC_TIMEZONE`	Set system and workflow scheduling defaults to Asia/Manila.
`N8N_SECURE_COOKIE: "false"`	Allows this loopback-only HTTP exercise in Safari. Not a public deployment setting.
`N8N_RUNNERS_MODE: internal`	Runs the trusted JavaScript exercises within this single-user lab setup.
mock-api `--hostname 0.0.0.0`	Allows the n8n container to reach Mockoon inside the Docker network.

Setting	Why it is here
`127.0.0.1:3000:3000`	Publishes the mock API/website only through the Mac-side loopback address.
`./mockoon:/data:ro`	Lets Mockoon read your configuration and website files; `ro` means read-only source files.

The internal `0.0.0.0` listener and the Mac-side `127.0.0.1` publication are different layers. Do not remove the Mac-side loopback restriction. Read-only source files do not prevent Mockoon from changing its in-memory records. The cookie and runner settings are lab choices, not a hardened multi-user production configuration. [R4, R5, R8, R9]

The `latest` tags are moving image tags. They make the first download straightforward but are not reproducible version pins. After the lab works, record the versions and use fixed tags before any controlled update; Chapter 20 explains this.

Step 5 - Validate the configuration

In the same Terminal folder, run:

```
docker compose config
```

Expect a normalized configuration. A message about a missing configuration file normally means you are in the wrong folder. A YAML parsing error normally means indentation or filename/content damage. Restore the unchanged file from the ZIP instead of guessing at several edits.

5. Start n8n and open it in Safari

Step 1 - Start only the n8n service first

Run:

```
docker compose up -d n8n
```

Docker downloads the image when necessary, creates the project's network and volume, and starts the n8n container. `-d` means it runs in the background, so the Terminal prompt returns. Keep Docker Desktop open. The first download may take several minutes. [R9]

Step 2 - Check state, port and startup logs

Run these separately:

```
docker compose ps
docker compose port n8n 5678
docker compose logs --tail=60 n8n
```

The port command should report `127.0.0.1:5678`. In Docker Desktop, expand the `taskflow-local` project and select its n8n container. The Logs tab corresponds to the logs command. The exact container name may include the project and a number; use Compose service commands rather than guessing that name.

The container can be running before the application finishes starting. Wait for the logs to indicate that n8n has started, then test the browser. A repeating startup error or a Restarting state needs attention; do not repeatedly create more containers.

Step 3 - Enter the address exactly

In Safari's address bar, type:

```
http://localhost:5678
```

Use `http`, not `https`, and include `:5678`. This address refers to your own Mac, not your n8n Cloud account. A second useful check is `http://127.0.0.1:5678`.

Step 4 - Create the local owner account

On the first successful visit, complete the owner-account form with your name, email and a unique password. This creates an account in this local instance. It does not import your cloud workflows or cancel a cloud subscription.

The basic lab does not require a paid n8n license. Registration for optional free Community features and a paid plan are different choices. Do not purchase a subscription just to proceed through these exercises. [R1]

Step 5 - Prove that n8n data persists

Create a workflow named `LAB - Storage check`, add Manual Trigger, and save it. Then run:

```
docker compose restart n8n
```

Wait for startup, reload Safari and confirm that the workflow remains. This is a simple persistence check, not a full disaster-recovery test. Never use volume deletion as part of this test.

Step 6 - Fix the specific Safari failure systematically

Observation	Next action
Port output is missing or only `5678/tcp` appears for an old container	Use the guide's Compose instance and check its port command; restarting the old container does not add a mapping.
Port already allocated	Stop the other container/application using 5678; keep its data. Then rerun `docker compose up -d n8n`.
n8n is Restarting	Read the last 60 log lines. Resolve the reported startup error before browser changes.
`localhost` fails but `127.0.0.1` works	Continue with the IP address and investigate local name resolution/proxy configuration separately.
A secure-cookie message appears	Confirm the lab's `N8N_SECURE_COOKIE` value, then run `docker compose up -d n8n`; a plain restart does not apply changed environment configuration.
The owner screen returns after prior work	Confirm the project and volume. Do not immediately create a second owner or delete data.

Milestone 1: stop here until the n8n editor opens and the storage-check workflow survives a restart. Docker is now installed and n8n is running locally. The next chapters add the fake server it will call.

6. Install Mockoon Desktop and learn the screen

Step 1 - Download the local application

Open Mockoon's official download page: https://mockoon.com/download/. Under macOS, choose Silicon for an Apple chip, Intel for an Intel Mac, or the Universal build. This guide uses **Mockoon Desktop**, not a paid cloud workspace. [R3]

Step 2 - Install and open it

Open the downloaded disk image, drag Mockoon to Applications, and launch it from Applications. Follow normal macOS confirmation prompts for the official download. Do not install Homebrew, npm or an extra Node.js runtime just for this lab; Docker supplies the headless server later.

Step 3 - Take the interface tour

Use the initial tour, or open **Help -> Take the tour** when available. An **environment** is one mock API project. It includes a listening port, routes, response settings and data buckets. A route is one path or resource inside that environment. [R10]

Area	What you do there
Environment list	Select which API project you are editing.
Routes list	Select paths such as `/health` or `/certificates`.
Route editor	Set a normal route's method, path and response.
Response body	Enter the JSON/text a normal route returns.
Data tab	Create named data buckets for changeable collections.
Environment settings	Inspect port, hostname, prefix and server options.
Play/stop control	Start or stop the selected Desktop server.
Logs view	Inspect requests received by that Desktop server.

Button placement can change between versions. Use the names and the official tour to identify the controls. This manual's numbered steps describe the intended operation, not a requirement that every icon occupy the same pixel.

Step 4 - Distinguish editing from running

Mockoon Desktop can run a server itself, but the main TaskFlow API in this edition runs through Mockoon CLI in Docker. The Desktop app edits the same environment file. It is not a cloud account, and the CLI is not a different mock API product you must subscribe to.

Desktop's green play button is used in the short practice exercise below. After that exercise, stop it. During the main TaskFlow workflows, leave the TaskFlow Desktop environment stopped and start the `mock-api` Docker service instead.

Desktop's Logs view does not show requests made to a separate Docker CLI process. For the main lab, use Docker's mock-api logs and the fresh response/website records as evidence.

7. Make one small Mockoon API yourself

This exercise teaches the controls before you open the larger supplied environment. It uses port **3001**, not the main lab's 3000, so the examples cannot be confused with the full inventory.

Step 1 - Create the practice environment

In Mockoon, choose **New environment**. Save the file as `practice/mockoon-basics.json` inside `TaskFlow-Local-Lab`. Name the environment `Mockoon Basics`. Set its port to `3001`, leave the endpoint prefix empty, keep proxy mode off, and set the listening hostname to `127.0.0.1` in the environment settings. [R10, R11]

An environment file is the saved recipe. It is not the same thing as a running API or its current live records.

Step 2 - Create a normal GET route

Use the route add menu and select **HTTP route**. Set method **GET** and path **health**. The public path becomes `/health`; do not paste a full URL into the path field. Set status **200**, response type/body to inline text, and paste:

```
{"ok":true,"message":"My first Mockoon API"}
```

Add a response header with name `Content-Type` and value `application/json`. Start the environment using the green play control.

Step 3 - Read the route in Safari

Open:

```
http://localhost:3001/health
```

Expect the JSON from Step 2. Safari may show formatted JSON or plain text; both are acceptable. A root-page 404 at `/` says nothing about `/health`, so use the full route address.

Change the message in Mockoon's response editor and repeat the request. Where necessary, stop/start the Desktop server to apply the change. This demonstrates a fixed response: it returns the configured body, not a stored task created by another request.

Step 4 - Create a data bucket

Click the **Data** tab. Click its plus/add control. Name the new bucket `practice-certificates`. Paste the complete array below into the bucket's content editor. The same text is in `practice/certificate.json` in the ZIP. [R12]

```
[
  {
    "id":"practice-1",
    "name":"practice-api",
    "owner":"Lab Operator",
    "expiresAt":"2030-01-01T00:00:00.000Z"
  }
]
```

Square brackets mean a list. Braces mean one object. The `id` identifies the row. Use straight double quotes and no trailing comma. The 2030 date is only a simple fixed example for learning the editor, not one of the real exercise dates.

Step 5 - Link that bucket to a CRUD route

Return to the route area. Choose **Add route -> CRUD route**, not HTTP route. Set path `certificates`. In its default response, select the `practice-certificates` data bucket and keep the CRUD key as `id`.

You do not manually add GET, POST and PATCH routes for that same collection. The CRUD route supplies those operations. Its method and status controls differ from a normal route because Mockoon chooses them for each operation. [R6]

Step 6 - Test a real change between requests

Keep the practice Desktop server running. In Terminal, run:

```
curl http://localhost:3001/certificates
```

Then change only the owner:

```
curl -X PATCH http://localhost:3001/certificates/practice-1 \
  -H 'Content-Type: application/json' \
  -d '{"owner":"Database"}'
```

Now read again:

```
curl http://localhost:3001/certificates/practice-1
```

Expect the same `id` with owner `Database`. The backslashes in the multi-line command continue it on the next line; paste the whole block. `curl` is an HTTP client in Terminal, not a Docker command.

Step 7 - Learn the reset behavior deliberately

Stop and start **Mockoon Basics** in Desktop. Read the record again. Its owner should return to the configured starting value `Lab Operator`. Live CRUD changes did not rewrite the saved bucket definition. This reset behavior matters during long rotation exercises. [R6, R12]

Checkpoint: You created a route, created a bucket, connected a CRUD resource, changed a row, verified the change, and observed what restart resets. Stop Mockoon Basics now. Leave the file in `practice`; it is not part of the main TaskFlow API.

8. Open and run the complete TaskFlow API

Step 1 - Open the supplied environment correctly

In Mockoon Desktop, choose **Open environment** and select:

```
TaskFlow-Local-Lab/mockoon/taskflow.mockoon.json
```

Do not use OpenAPI import. This is Mockoon's own environment format. If the app offers to migrate an older format, allow the format update and keep the file in its original folder. Use a compatible current CLI image when starting it. [R13]

Select `TaskFlow Local Lab` in the environment list. Check port `3000`, empty prefix, no proxy, and a Desktop listening hostname of `127.0.0.1`. Keep its Desktop server **stopped**. The Compose command overrides the listener only inside the Docker container.

Step 2 - Inspect the ten data collections

Open Data. The following bucket names should exist. Their starting values are empty arrays. The website initializes the exercise records with dates relative to the day you begin, instead of stale calendar dates.

Collection	Contents	Initial count after website reset
`crypto-assets`	Inventory and optional SSH/TLS observations	8
`certificates`	Expiry, owner and mock fingerprint	5
`jwt-keys`	Signing/verification metadata	1
`services`	Scoped PQC evidence	3
`vendors`	Invented vendor statements and review dates	6
`secrets`	Credential version/rotation metadata, never secret values	2
`tasks`	Findings and closure evidence	0
`rotation-requests`	Simulated approval and lifecycle records	0
`notifications`	Mock notification records, not delivered messages	0
`ci-checks`	Simulated commit-specific decisions	0

For any bucket, select it, inspect its name and value, then select the matching CRUD route and confirm that the response references that bucket with CRUD key `id`. You learned this connection in Chapter 7.

Step 3 - Understand the route catalog

Each collection exposes `GET /collection`, `POST /collection`, `GET /collection/ID`, `PATCH /collection/ID`, and other documented CRUD operations. `PUT /collection` replaces the full collection; the website uses that for reset and restore. Do not use it when you intend to change only one row. [R6]

The remaining routes serve the website and support files:

Route	Type and response
`GET /`	File `./index.html`, as body, `text/html; charset=utf-8`
`GET /styles.css`	File `./styles.css`, `text/css; charset=utf-8`
`GET /app.js`	File `./app.js`, `application/javascript; charset=utf-8`
`GET /seed-data`	File `./seed-data.json`, `application/json`
`GET /events/ci-risky`	File `./ci-risky.json`, `application/json`
`GET /events/ci-clean`	File `./ci-clean.json`, `application/json`
`GET /events/incident`	File `./incident.json`, `application/json`
`GET /health`	Inline JSON confirming this lab
`GET /simulate-error`	HTTP 503 for the error-handling exercise

Step 4 - Know how to recreate a file route

Select the root GET route. Its path is empty or shown as `/`. Its response body type is **File**, path `./index.html`, status 200, and **Send as body** is enabled. The Content-Type is HTML. File/body templating is disabled. Missing-file fallback to 404 is enabled in the supplied file.

To recreate another file route, add a normal HTTP route, choose GET, enter the path from the table, select File, and enter the relative file path and matching Content-Type. Relative paths are resolved from the environment file, which is why all website files stay in the `mockoon` folder. [R14]

Step 5 - Start the Docker mock server

The CLI image runs the saved environment without opening another Desktop window. [R26]

Return to Terminal in the project folder. Confirm the TaskFlow Desktop server is stopped, then run:

```
docker compose up -d mock-api
docker compose ps
docker compose port mock-api 3000
docker compose logs --tail=60 mock-api
```

Expect the port to be `127.0.0.1:3000`. In Safari, open:

```
http://localhost:3000/health
```

Expected body:

```
{"ok":true,"environment":"taskflow-local","simulated":true}
```

If the image cannot start because its environment format is too old/new, open and save the environment in a compatible Desktop release, then use a matching CLI release. Do not change arbitrary schema fields by hand.

Step 6 - Apply configuration changes deliberately

Changing the Desktop file does not make a separate running CLI process magically reload every setting. To apply a saved route/configuration change, run:

```
docker compose restart mock-api
```

This resets its live data. Export a snapshot first, then restore it afterward, or initialize a new exercise. Editing a website file is different: it is served from the mounted folder, so a browser reload normally shows the changed file without restarting the server.

Checkpoint: Both Docker services are running, the health route identifies this edition, and no Desktop server is competing for port 3000.

9. Set up and use the mock website in detail

9.1 What this website is

It is the supplied TaskFlow dashboard, not Mockoon's website and not an n8n page. Mockoon serves its files locally. You do not buy a domain, configure Cloudflare, create a Supabase project, or run a Python web server.

`index.html` defines the page's headings and controls. `styles.css` controls appearance. `app.js` sends requests and displays results. `seed-data.json` contains starting records. You can inspect these files in a plain-text editor, but they already contain the required code.

Step 1 - Open the website through its server

In Safari, open:

```
http://localhost:3000
```

Do not double-click `index.html` in Finder. That opens a `file://` page, and its `/tasks`-style URLs no longer refer to the intended server. No API base URL needs to be pasted into this website: its relative paths use the same origin as the page.

Checkpoint: You see the heading **TaskFlow Security Lab**, its mock-data warning, and controls headed Initialize and inspect, Record editor, Test events, and Snapshots. Raw JSON means you opened an API collection instead of `/`. A file download instead of a page means the root response's Content-Type or Send as body setting is wrong.

Step 2 - Initialize the fake records

Click **Initialize / reset demo**. Read the confirmation: reset replaces all ten collections and removes previous lab tasks and rotation progress. Confirm only when no workflow is running.

The browser fetches `/seed-data`, converts markers such as `@days:6` into timestamps six days from now, and sends each collection to its CRUD endpoint. You do not manually calculate dates or type dozens of rows into the Data tab.

Checkpoint: The cards show 8 inventory assets and 5 certificates. Tasks and rotation requests start empty. The five certificate scenarios are roughly 6, 25, 10 and 90 days ahead, plus one deliberately invalid date. This is intentional test data, not evidence of a damaged installation.

Step 3 - Inspect a collection and one row

Scroll to **Inspect or edit a record**. In Collection, select `certificates`. In Record ID, select `cert-001`. Click **Load selected record**. Its JSON appears in the large editor.

Open **Show full collection JSON** to see all certificates. The full-list view is for inspection; the editable box contains the one selected row. The two are not the same target.

Look at `id`, `name`, `owner`, `expiresAt` and `fingerprint`. The fingerprint is a lab label, not a computed hash of a real certificate. Leave `id` unchanged so the save targets the same row.

Step 4 - Change a certificate without guessing a date

With `cert-001` loaded, click **Set expiry: 25 days**. The editor's timestamp changes, but the API has not been updated yet. Click **Save edited record**.

The page sends a PATCH for the loaded ID and then performs a fresh GET to check the saved fields. Expect a message saying it was saved and verified. Click **Refresh all**, load `cert-001` again, and confirm the new timestamp is still present.

Checkpoint: You can explain the difference between changing a value in the text box and saving it to the API. The new value survives a separate GET while Mockoon remains running.

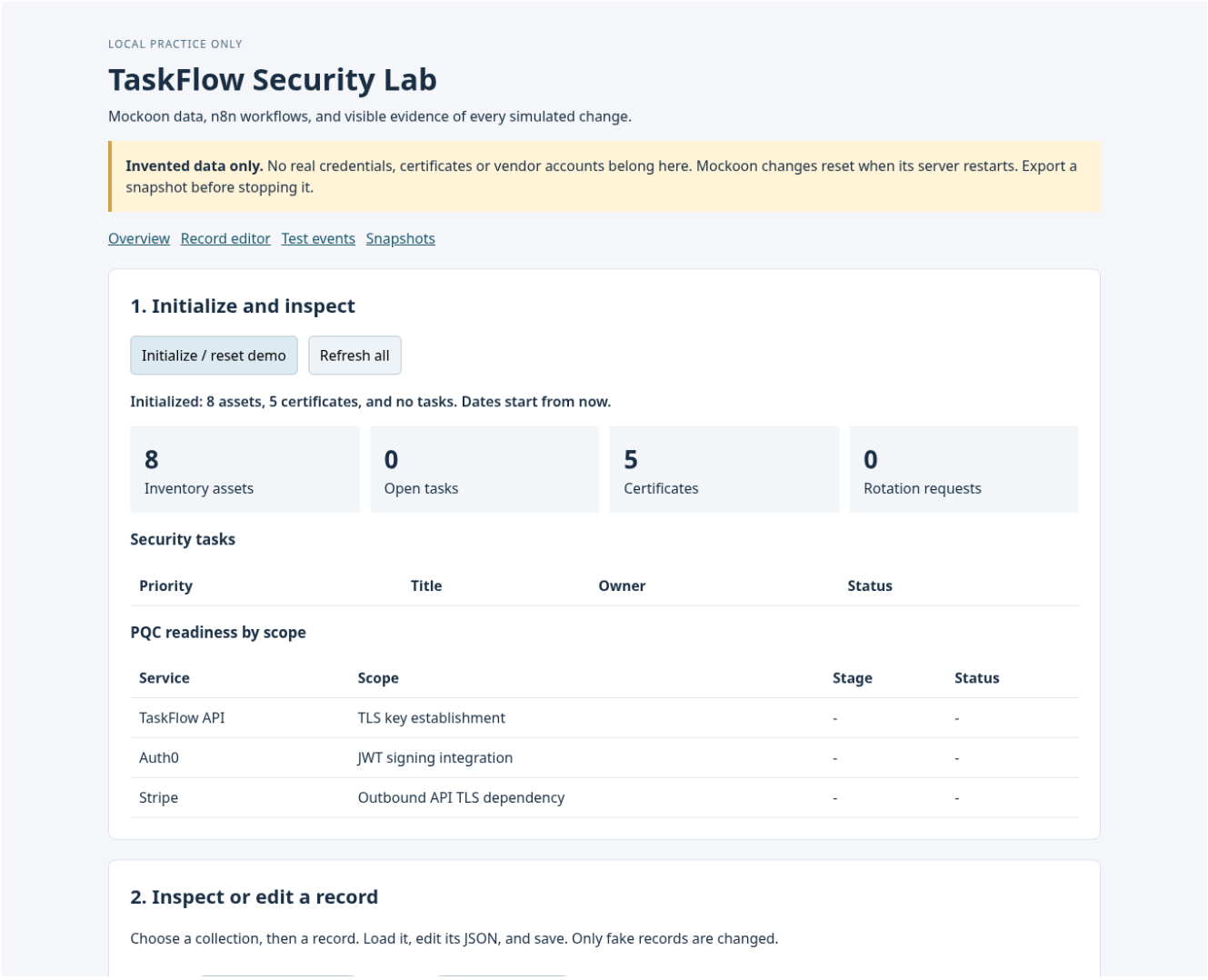
Step 5 - Make a manual field change

Change only the owner value from `Platform` to `Lab Operator`, keeping the double quotes. Save, refresh and load again. A JSON error normally means a missing quote, missing comma, or extra comma before a closing brace. Correct the editor before retrying.

For a boolean, use `true` or `false` without quotes. `"false"` is a string, not the boolean false. For a missing value use `null`, not a blank word. The website does not let you change a record ID through this editor.

Step 6 - Return to the known test baseline

After the practice edits, click **Initialize / reset demo** again. Confirm the counts and reload cert-001; its expiry should be approximately six days away. This removes the trial edits so the workflow expected-results tables remain meaningful.



The supplied dashboard after initialization. This illustration uses the kit's invented records; the readiness results populate when Workflow 4 runs.

9.2 Use the website alongside every workflow

After running a workflow in n8n, return here and click **Refresh all**. The page does not poll continuously. Use the tasks table to see titles, owners, priorities and statuses. Choose `notifications` in the record editor to inspect mock alerts. A notification record is not an email or an on-call page.

For JWT and secret exercises, choose `rotation-requests` to inspect the request. A row appears only after the detector creates it. Load it, inspect the target, then change approval fields only at the step where the manual asks you to simulate approval. Anyone who can edit this local mock data can change those fields; they are not a real authorization system.

9.3 Send a test event from the website

This control becomes useful in Workflows 7 and 8. Do not press it before creating the receiver in n8n.

Choose **CI: crypto patterns**, **CI: no configured pattern**, or **Incident: certificate drill**. The page loads the matching local JSON file. Copy the exact n8n Test URL into the webhook URL field. Enter your practice-only X-Lab-Token in the password field; it is not saved in browser storage or the starter files.

In n8n, click **Listen for test event**. Return to the page and click **Send mock event**. Inspect the HTTP result and n8n's execution. The website restricts this sender to this lab's local addresses and paths.

The page and n8n have different ports, so this is a cross-origin browser request. In the receiving Webhook node, set **Allowed Origins (CORS)** to the page's origin, normally `http://localhost:3000`; include `http://127.0.0.1:3000` when using that address. CORS is not authentication. If a browser/preflight restriction still interferes, use the Terminal command in Appendix B and check n8n before retrying; do not disable browser security or switch fetch to `no-cors`. [R15]

9.4 Save your mock progress before stopping

Click **Export fresh snapshot**. The website reads the collections again, then downloads a JSON file named `taskflow-snapshot-...json`. In Safari, find it in Downloads and move it to a clearly named folder for this lab.

When you restart Mockoon later, its buckets are empty again. To resume, choose that file under Snapshots and click **Restore selected snapshot**. Stop workflows first. Restore replaces all collections and keeps the old timestamps; it does not make them relative to today. A certificate may therefore be closer to expiry than when you exported it.

To begin fresh instead, use Initialize / reset demo. Do not initialize after restoring a snapshot unless you intend to erase the restored progress. A failed restore may have replaced only some collections; inspect the error and repeat a known-good restore before running workflows.

9.5 Common website changes

To change the visible title, open `mockoon/index.html` in a plain-text editor, edit the text inside the existing h1 element, save, and reload Safari. To change a seed record for future resets, edit the appropriate object in `mockoon/seed-data.json`, preserve valid JSON, save, then explicitly reset through the website.

Do not paste n8n credentials into `app.js`, and do not publish this website or its unauthenticated mock API on the internet. The setup is deliberately local.

> Milestone 2: the website loads, the demo initializes, a PATCH survives read-back, and you can export and restore a snapshot. The unfamiliar software is now set up; the remaining work is primarily n8n automation.

10. Connect n8n and build one reusable task handler

10.1 Use one HTTP convention throughout

For every Mockoon HTTP Request node, use the following settings unless a step says otherwise:

Field	Value
Base URL	`http://mock-api:3000`
Authentication	None, for this local mock API only
Response Format	JSON
Include Response Headers and Status	On

Field	Value
Never Error	Off
Timeout	10000 milliseconds
GET requests	Send Body off
POST/PATCH requests	Send Body on, JSON, Using JSON

A short path such as `GET /certificates` in the remaining chapters means an HTTP Request with method GET and URL `http://mock-api:3000/certificates`. It is not a Terminal command.

A full-response GET returns a wrapper such as `{"body":[...],"headers":{...},"statusCode":200}`. The records are in `$json.body`. After splitting a collection, a record's fields are directly in `$json`. An expression field containing `{{ ... }}` evaluates JavaScript; a literal JSON field does not evaluate arbitrary JavaScript. [R16]

Step 1 - Test the container-to-container connection

Create `LAB - API connection check` with Manual Trigger and an HTTP Request named `Get health`. GET `http://mock-api:3000/health` using the settings above. Execute it and inspect `body.ok`.

If Safari can open the mock website but this node fails, check that both services belong to the `taskflow-local` project and that the node uses `mock-api`, not localhost. If the error explicitly says the destination resolves to a restricted IP, see the narrowly scoped SSRF troubleshooting entry in Appendix C.

Step 2 - Split a collection in a predictable way

Change or duplicate the request to GET `/certificates`. Initialize the website first. Add a JavaScript Code node named `Split collection`, mode **Run Once for All Items**, and paste this script, also supplied in the named file:

Copyable file: `code/00-split-collection.js`

```
// n8n Code node: JavaScript, Run Once for All Items.
// Upstream HTTP Request: JSON response + Include Response Headers and Status ON.
const rows = $input.first().json.body;
if (!Array.isArray(rows)) throw new Error('Expected an array in response.body');
return rows.map(row => ({ json: row, pairedItem: { item: 0 } }));
```

Expect five output items. Most n8n nodes already process each incoming item; an explicit Loop Over Items node is needed only for batching, pacing or controlled sequencing, not simply because there are several rows.

10.2 Build LAB 00 - Upsert Security Task

An **upsert** updates an existing finding or inserts a new one. Here it is a lookup followed by a write, suitable for sequential practice. It is not a concurrency-safe database transaction. The helper is a reusable component for the eight workflows, not an additional security use case.

Step 1. Create `LAB 00 - Upsert Security Task`. Add **Execute Sub-workflow Trigger**, also displayed as **When Executed by Another Workflow**. Set Input data mode to **Accept all data**. The next node validates that data. [R17]

Step 2. Add Code `Validate task`, mode **Run Once for Each Item**:

Copyable file: **`code/00-validate-task.js`**

```
// Run Once for Each Item. Input is one proposed task, not an array.
const t = $json;
for (const k of ['dedupeKey','ruleId','assetId','title','owner','reason']) {
  if (typeof t[k] !== 'string' || !t[k].trim()) {
    throw new Error('Missing task field: ' + k);
  }
}
if (!['LOW','MEDIUM','HIGH','CRITICAL'].includes(t.priority)) {
  throw new Error('Invalid priority');
}
return { json: { ...t, observedAt: t.observedAt || new Date().toISOString() } };
```

A task's `dedupeKey` is the stable identity of the finding, for example `certificate-expiry:cert-001`. Its stored `id` is assigned to the row by Mockoon. Do not put the current timestamp into the dedupe key or every run will look like a new finding.

Step 3. Add HTTP Request `Find task`. GET `/tasks`. Enable Send Query Parameters and add name `dedupeKey_eq`, value expression `{{ $json.dedupeKey }}`. Mockoon's equality filter returns matching rows as an array. Full-response mode keeps an empty array visible for the insert branch. [R6]

Step 4. Add Code `Choose write`, mode **Run Once for Each Item**:

Copyable file: **`code/00-choose-task-write.js`**

```
// Run Once for Each Item. Upstream node is Find task (full HTTP response).
// The earlier Code node MUST be named Validate task.
const t = $('Validate task').item.json;
const rows = $json.body;
if (!Array.isArray(rows)) throw new Error('Task lookup is not an array');
if (rows.length > 1) throw new Error('Duplicate dedupeKey; repair lab data');
const old = rows[0];
const rank = { LOW: 0, MEDIUM: 1, HIGH: 2, CRITICAL: 3 };
const body = {
  ...t, evidence: {...(old?.evidence || {}), ...(t.evidence || {})},
  status: 'open', lastSeenAt: t.observedAt,
  createdAt: old ? old.createdAt : t.observedAt
};
// A previously closed task reopens only because a detector found it again.
const notify = !old || old.status === 'closed' || rank[t.priority] > rank[old.priority];
return { json: { existingId: old ? old.id : null, body, notify } };
```

The script refuses multiple matching records. It preserves saved evidence while refreshing observations. A previously closed task reopens when a detector reports the condition again. A new, reopened or higher-priority finding requests a notification; an unchanged repeat does not.

Step 5. Add an IF testing whether `existingId` is not empty. On true, PATCH the existing task with the following URL expression:

```
{{ 'http://mock-api:3000/tasks/' +
   encodeURIComponent($json.existingId) }}
```

On false, POST to `http://mock-api:3000/tasks`. For both nodes, set the entire JSON body to the expression:

```
{{ $json.body }}
```

Step 6. Connect both write branches directly to one HTTP Request named `Verify saved task`. Only the branch that ran reaches it. GET the saved row using:

```
{{ 'http://mock-api:3000/tasks/' +
   encodeURIComponent($json.body.id) }}
```

Step 7. Add Code `Check saved task`, mode **Run Once for Each Item**:

```
const saved = $json.body;
const expected = $('Choose write').item.json.body;
if (!saved || saved.dedupeKey !== expected.dedupeKey ||
    saved.priority !== expected.priority || saved.status !== 'open') {
  throw new Error('Task read-back did not match the requested write');
}
return {json: saved};
```

Step 8. Add an IF whose boolean expression is `{{ $('Choose write').item.json.notify }}`. On true, POST `/notifications` with:

```
{{ ({
  taskId: $json.id,
  priority: $json.priority,
  owner: $json.owner,
  message: $json.title + ': ' + $json.reason,
  createdAt: new Date().toISOString()
}) }}
```

Connect that POST and the IF's false branch to Code `Return saved task`, mode **Run Once for Each Item**:

```
return {json: $('Check saved task').item.json};
```

Save the helper. Its last output is the task, not the notification response.

10.3 Test the helper before any detector

Create a separate test workflow with Manual Trigger, an Edit Fields node in JSON mode, and **Execute Sub-workflow**. In Edit Fields paste:

```
{
  "dedupeKey":"smoke:task-001",
  "ruleId":"smoke",
  "assetId":"cert-001",
  "title":"Practice task creation",
  "priority":"HIGH",
  "owner":"Lab Operator",
  "reason":"Testing the reusable task handler"
}
```

Select `LAB 00 - Upsert Security Task` in the Execute Sub-workflow node. Set **Run once for each item** and **Wait for Sub-Workflow Completion** on. Use those same settings whenever a later chapter says **call LAB 00**. [R17]

Run twice with unchanged input. Refresh the website and inspect tasks and notifications. Expect one task, one initial notification and the same task ID. Change priority to CRITICAL and run again; expect the same task plus one escalation notification.

Checkpoint: Duplicate prevention and read-back work. Reset the website to clear this test; saved n8n workflows remain. Keep schedules disabled while building the remaining workflows.

10.4 Handling one result from a filtered lookup

Several later steps say to require exactly one matching task or request. After the filtered GET, use this Code node in **Run Once for Each Item** mode:

```
const rows = $json.body;
if (!Array.isArray(rows) || rows.length !== 1) {
  throw new Error('Expected exactly one record; inspect the lookup');
}
return {json: rows[0]};
```

Output is one record. Use `$json.id` next; missing or duplicate matches remain errors.

Part III - Build the eight workflows

11. Workflow 1 - Crypto Asset Discovery

Goal: ingest mock inventory, classify its algorithm labels, record the assessment and create review tasks. Reading a prepared list simulates discovery results; it does not scan your Mac, AWS account or code repositories.

Step 1 - Fetch and split

Create `LAB 01 - Crypto Asset Discovery`. Connect Manual Trigger -> HTTP Request `Get assets` -> Code `Split collection`. GET `/crypto-assets` and use the Chapter 10 splitter. After initialization, expect eight items.

Step 2 - Classify the records

Add Code `Classify assets`, mode **Run Once for All Items**:

Copyable file: `code/01-classify-assets.js`

```
// Run Once for All Items; place after Split collection.
const publicKey = /RSA|ECDSA|ECDH|ECC|ED25519|ED448|X25519|X448|^(RS|PS|ES)(256|384|512)$/;
const notPublicKey = /^(AES|SHA|HMAC)/;
const pqcName = /ML-KEM|ML-DSA|SLH-DSA/;
const now = new Date().toISOString();
return $input.all().map((item,i) => {
  const a = item.json;
  const alg = String(a.algorithm || '').trim().toUpperCase();
  let assessmentStatus, quantumRisk, finding = null;
  if (publicKey.test(alg)) {
    assessmentStatus = 'PQC assessment required'; quantumRisk = true;
  } else if (notPublicKey.test(alg)) {
    assessmentStatus = 'Separate symmetric/hash assessment'; quantumRisk = null;
  } else if (pqcName.test(alg)) {
    assessmentStatus = 'PQC-labelled; verify implementation evidence'; quantumRisk = null;
  } else {
    assessmentStatus = 'Unknown algorithm; investigate'; quantumRisk = null;
  }
  if (quantumRisk === true || assessmentStatus.startsWith('Unknown')) {
    finding = {
      dedupeKey: 'crypto-review:' + a.id, ruleId: 'crypto-review',
      assetId: a.id, title: 'Review ' + a.service + ' cryptography',
      priority: 'MEDIUM', owner: a.owner || 'Unassigned',
      reason: assessmentStatus + ' (' + (alg || 'missing') + ')',
      observedAt: now
    };
  }
  return { json: {...a, assessmentStatus, quantumRisk, lastSeenAt: now, finding},
    pairedItem: {item:i} };
});
```

The pattern list includes common RSA/ECC-related names and JWT identifiers. Matching only the letters ECC would miss labels such as ECDSA. An unfamiliar or missing label becomes an investigation task. AES/hash labels follow a separate assessment category; they are not automatically declared safe for every use.

Step 3 - Save the assessment

Add HTTP Request `PATCH inventory`. Method PATCH, URL expression:

```
{{ 'http://mock-api:3000/crypto-assets/' +
   encodeURIComponent($json.id) }}
```

JSON body expression:

```
{{ ({
  assessmentStatus: $json.assessmentStatus,
  quantumRisk: $json.quantumRisk,
  lastSeenAt: $json.lastSeenAt
}) }}
```

Step 4 - Send only findings to the helper

After the PATCH, add an IF with boolean expression:

```
{{ Boolean($('Classify assets').item.json.finding) }}
```

On true, add Code in **Run Once for Each Item** mode:

```
return {json: $('Classify assets').item.json.finding};
```

Call LAB 00. The false branch ends without a task. The reference to `Classify assets` deliberately retrieves the original item after the HTTP node replaced the current output with its response.

Step 5 - Check the outcome

Refresh the website. Expect seven crypto-review tasks: six recognized public-key labels plus one unknown label. All eight inventory rows receive an assessment. The AES example receives no task from this particular detector.

Run again. The task count should not grow. Change a label to an unfamiliar value and confirm it is investigated, not marked safe. After manual tests, add a daily Schedule Trigger and set the workflow timezone deliberately. Publish only while you intend it to run.

Done when: classifications are visible, unknown labels are not ignored, tasks have owners, and repeated observations update the same findings.

12. Workflow 2 - Certificate Expiry Automation

Goal: calculate exact remaining time, create a severity-appropriate task and handle bad dates explicitly.

Step 1 - Build the chain

Create `LAB 02 - Certificate Expiry`. Connect Manual Trigger -> GET `/certificates` -> Split collection -> Code `Evaluate certificates` -> Execute Sub-workflow LAB 00. Set the evaluator to **Run Once for All Items**:

Copyable file: `code/02-certificate-findings.js`

```
// Run Once for All Items; input items are individual certificates.
const now = Date.now(), day = 86400000, out = [];
function utcMs(s) {
  if (typeof s !== 'string' || !/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(\.\d{3})?Z$/.test(s))
return NaN;
  const ms = Date.parse(s);
  if (!Number.isFinite(ms)) return NaN;
  return new Date(ms).toISOString().replace('.000Z','Z') === s.replace('.000Z','Z') ? ms : NaN;
}
$input.all().forEach((item,i) => {
  const c = item.json, expiry = utcMs(c.expiresAt);
  const remaining = (expiry-now)/day;
  let priority, ruleId, reason;
  if (!Number.isFinite(expiry)) {
    priority='HIGH'; ruleId='certificate-data-quality';
    reason='expiresAt is missing or is not a valid UTC ISO timestamp';
  } else if (remaining >= 30) { return; }
  else {
    ruleId='certificate-expiry';
```

```
    priority=remaining<7?'CRITICAL':remaining<14?'HIGH':'MEDIUM';
    reason=remaining<=0?'Certificate is expired':
      'Certificate expires in '+Math.ceil(remaining)+' displayed day(s)';
  }
  out.push({json:{dedupeKey:ruleId+':'+c.id, ruleId, assetId:c.id,
    title:'Review certificate: '+c.name, priority,
    owner:c.owner||'Unassigned', reason, observedAt:new Date(now).toISOString(),
    evidence:{expiresAt:c.expiresAt,daysRemaining:Number.isFinite(remaining)?remaining:null}},
    pairedItem:{item:i}});
});
return out;
```

This code emits only actionable findings. It validates UTC timestamps. It compares the unrounded remaining time to the policy and rounds upward only for a readable display. A certificate with 6.9 days remaining is in the below-seven-day tier even if a display says seven days.

Step 2 - Understand the expected results

Actual remaining time	Priority	Fixture immediately after reset
Expired or below 7 days	CRITICAL	cert-001, approximately 6 days
At least 7 and below 14	HIGH	cert-003, approximately 10 days
At least 14 and below 30	MEDIUM	cert-002, approximately 25 days
At least 30	No expiry task	cert-004, approximately 90 days
Invalid/missing timestamp	HIGH data-quality task	cert-005, `not-a-date`

Expect four tasks. The expiry and data-quality rules use different dedupe keys from Workflow 1. Do not combine every issue about the same certificate into one ambiguous task.

Step 3 - Test escalation through the website

Load cert-002 in the website, set expiry to six days, and save. Rerun the workflow. Its existing expiry task should escalate from MEDIUM to CRITICAL and create an additional notification. An unchanged repeat should not create another initial alert.

At exactly 7 days, the policy tier is HIGH; exactly 14 is MEDIUM; exactly 30 produces no expiry task. Test boundaries with a fixed clock in a controlled test rather than relying on a live clock remaining at an exact boundary between clicks.

Step 4 - Simulate renewal and verify it

Load cert-001, set expiry to 90 days, and change its fingerprint to a different `LAB-...` label. Save, then load again to obtain a fresh observation. Only after the fingerprint and expiry checks pass should you close the related `certificate-expiry:cert-001` task, recording `verifiedAt` and a closure reason.

Chapter 18 provides a complete n8n verification/closure sequence. For now, use the website to inspect both the certificate and its task. Do not close its separate crypto-review task: renewing an RSA certificate did not migrate its algorithm.

Done when: all four baseline findings are correct, the invalid date creates work, escalation is visible, and simulated renewal is verified before closure. Reset before the next baseline tests when your edits would change their expected results.

13. Workflow 3 - JWT Key Rotation Simulator

Goal: model approved staging, signing cutover, verification overlap and retirement. No private key is generated and no JWT is signed. The records describe a lifecycle, not functioning cryptographic material.

A `kid` is a key identifier. A JWKS is a set of public keys used by verifiers. In this lab, `published`, `useForSigning` and `useForVerification` are only metadata flags. Replacing an RS256 key with another RS256 key is a key rotation, not a PQC migration. [R19]

Step 1 - Detect an overdue signing key

Create `LAB 03A - Detect JWT Rotation`: Manual Trigger -> GET `/jwt-keys` -> Split collection -> Code `Find overdue JWT keys` -> LAB 00. Use **Run Once for All Items**:

Copyable file: `code/03-jwt-findings.js`

```
// Run Once for All Items. No key material is read or generated.
const now=Date.now(), out=[];
$input.all().forEach((item,i)=>{
 const k=item.json;
 if(!k.useForSigning)return;
 const created=Date.parse(k.createdAt), policy=Number(k.rotationPolicyDays);
 const bad=!Number.isFinite(created)||created>now||(!Number.isFinite(policy)||!(policy>0));
 const age=(now-created)/86400000;
 if(!bad&&age<=policy)return;
 const ruleId=bad?'jwt-data-quality':'jwt-rotation';
 out.push({json:{dedupeKey:ruleId+':'+k.id,ruleId,assetId:k.id,
  title:'Review signing key: '+k.id,priority:'HIGH',owner:k.owner||'Unassigned',
  reason:bad?'Invalid key age or rotation policy':'Signing key exceeds lab age policy',
  observedAt:new Date(now).toISOString(),requestId:'rotate-'+k.id,
  evidence:{ageDays:bad?null:age,rotationPolicyDays:policy}},pairedItem:{item:i}});
});
return out;
```

After reset, jwt-001 is approximately 100 days old under a 90-day policy. Expect one HIGH rotation task. A data-quality task must not authorize a rotation.

Step 2 - Create a pending request, not an automatic approval

After LAB 00, add an IF requiring `ruleId` to equal `jwt-rotation`. On true, GET `/rotation-requests` with query `id_eq` = `rotate-jwt-001`. If `$json.body.length` is zero, POST the following body to `/rotation-requests`. If it is one, leave the existing approval/progress untouched. More than one is an error to inspect.

Copyable file: `examples/rotation-request.json`

```
{
  "id": "rotate-jwt-001",
  "kind": "jwt",
  "assetId": "jwt-001",
  "newKeyId": "jwt-001-next",
  "status": "pending-approval",
  "approvedBy": null,
  "approvedAt": null,
  "simulated": true
}
```

This fixed ID belongs only to the supplied jwt-001 exercise. With multiple keys you would derive a stable request ID for each target and rotation cycle, rather than reusing this example indiscriminately.

Step 3 - Inspect, then approve manually

Refresh the website. Choose rotation-requests, load rotate-jwt-001 and inspect its target, kind and replacement ID. Create a small, separate n8n workflow `LAB - Approve JWT request` with Manual Trigger and PATCH `/rotation-requests/rotate-jwt-001`. Execute it only after that inspection. Body expression:

```
{{ ({
  status: 'approved',
  approvedBy: 'Lab Operator',
  approvedAt: new Date().toISOString()
}) }}
```

Reload the request in the website. It should now show the approval fields. This is a simulated human checkpoint, not tamper-proof authorization.

Step 4 - Gate the cutover

Create `LAB 03B - Approved JWT Cutover`. Begin with Manual Trigger -> GET `/rotation-requests/rotate-jwt-001`. Add an IF using this boolean expression:

```
{{ $json.body.status === 'approved' &&
   $json.body.kind === 'jwt' &&
   $json.body.assetId === 'jwt-001' &&
   $json.body.simulated === true &&
   Boolean($json.body.approvedBy) &&
   Number.isFinite(Date.parse($json.body.approvedAt)) }}
```

The false branch stops. On true, GET `/jwt-keys/jwt-001` and require `body.useForSigning` to be true. Then GET `/jwt-keys` with `id_eq=jwt-001-next` and require an empty response array. A candidate already present indicates a partial/prior run; stop and inspect it rather than creating another one.

Step 5 - Stage and publish the replacement metadata

PATCH `/rotation-requests/rotate-jwt-001` with `{"status":"running"}`. Then POST `/jwt-keys` with:

```
{{ ({
  id: 'jwt-001-next', keyId: 'jwt-001-next',
  service: 'TaskFlow Authentication', owner: 'Identity',
  algorithm: 'RS256', createdAt: new Date().toISOString(),
  rotationPolicyDays: 90, state: 'staged',
  published: true, publishedAt: new Date().toISOString(),
  useForSigning: false, useForVerification: true,
  consumerHealth: 'pass', environment: 'lab'
}) }}
```

Add a Wait node for five seconds to represent cache propagation. GET `/jwt-keys/jwt-001-next` afresh, then require `published=true`, `useForVerification=true`, and `consumerHealth=pass`. Stop on failure.

Step 6 - Switch the signer and keep overlap

PATCH `/jwt-keys/jwt-001-next` with:

```
{"state":"active","useForSigning":true}
```

Then PATCH `/jwt-keys/jwt-001` with:

```
{{ ({
  state: 'overlap', useForSigning: false,
  useForVerification: true, published: true,
  signingStoppedAt: new Date().toISOString(),
  retireAfter: new Date(Date.now() + 65000).toISOString()
}) }}
```

PATCH the request with `{"status":"overlap"}`. Read both key records and inspect the website: only the new key signs, but both verify. These separate metadata writes are not an atomic production signer switch.

For this exercise only, 60 seconds of token lifetime plus 5 seconds of clock-skew allowance gives a 65-second overlap after the old signer stops. Real cache, token-lifetime and revocation policies must determine a real overlap; five seconds and 65 seconds are not production recommendations.

Step 7 - Retire only after fresh checks

Create `LAB 03C - Retire JWT Key`. Use Manual Trigger and GET `/jwt-keys/jwt-001`. Continue only when:

```
{{ $json.body.state === 'overlap' &&
   Number.isFinite(Date.parse($json.body.retireAfter)) &&
   Date.parse($json.body.retireAfter) <= Date.now() }}
```

GET the new key and require signing and verification true and consumerHealth pass. PATCH the old key with:

```
{
  "state":"retired",
  "useForSigning":false,
  "useForVerification":false,
  "published":false
}
```

Read the old key again and verify those fields. Only then PATCH the request to completed. Find its task with GET `/tasks`, query `dedupeKey_eq=jwt-rotation:jwt-001`. Require exactly one row using Chapter 10.4, then PATCH `/tasks/ID` with:

```
{{ ({
  status: 'closed', verifiedAt: new Date().toISOString(),
  closureReason: 'Old JWT metadata retired after overlap and health checks'
}) }}
```

GET that task again and confirm closure. Leave Workflow 1's crypto-review task open.

Step 8 - Test refusal and rollback

Try cutover before approval: no key should be written. In a fresh exercise, arrange `consumerHealth=fail` on the staged candidate before the cutover check; signing must not switch. During overlap, a failed health check must prevent old-key retirement.

For a non-compromise lab rollback, manually restore the old key's signing and verification flags, disable signing on the new key, and mark the request rolled-back. Keep the task open. Never apply that rollback rule to a genuinely compromised old key.

Done when: approval is checked, publishing precedes signing, old verification survives overlap, failure prevents retirement, and read-back precedes closure.

14. Workflow 4 - PQC Readiness Scanner

Goal: calculate a scoped, evidence-based stage and identify the next missing action. A reference in a field is only a pointer; a real reviewer must inspect the referenced evidence.

Step 1 - Inspect the service inputs

In the website, choose services and load svc-api. Read `scope`, `crypto`, `vendorControlled`, `evidence`, and `blockers`. The LAB-prefixed evidence values are invented identifiers. The required order is inventory -> roadmap -> testEnvironment -> hybridTest -> production.

Step 2 - Calculate and save the stage

Create `LAB 04 - PQC Readiness`: Manual Trigger -> GET `/services` -> Split collection -> Code `Score readiness`, **Run Once for All Items**:

Copyable file: `code/04-readiness.js`

```
// JavaScript / Run Once for All Items.
const stages = ['inventory','roadmap','testEnvironment','hybridTest','production'];
const next = [
  'Identify cryptographic assets, scope and owner',
  'Document the vendor roadmap or internal migration decision',
  'Prepare a non-production test environment',
  'Run hybrid/PQC compatibility, performance and rollback tests',
  'Prepare an approved production change and verification plan'
];
return $input.all().map((item, i) => {
  const s = item.json, evidence = s.evidence || {};
  let score = 0;
  for (const field of stages) {
    if (typeof evidence[field] !== 'string' || !evidence[field].trim()) break;
    score++;
  }
  if (s.simulated === true) score = Math.min(score, 4);
  const blockers = Array.isArray(s.blockers) ? s.blockers : [];
  const readinessStatus = blockers.length ? 'Blocked' : score === 5 ? 'Migrated' :
    score === 0 ? 'Unknown' :
    s.vendorControlled && score < 4 ? 'Vendor-dependent' :
    score >= 3 ? 'Testing' : 'Inventoried';
  const finding = score < 5 || blockers.length ? {
    dedupeKey: 'readiness:' + s.id, ruleId: 'readiness', assetId: s.id,
    title: 'Plan next PQC step: ' + s.service,
    priority: 'MEDIUM', owner: s.owner || 'Security',
    reason: blockers.length ? blockers.join('; ') : next[score],
    observedAt: new Date().toISOString()
  } : null;
  return {json: {...s, score, readinessStatus, finding,
    assessedAt: new Date().toISOString()}, pairedItem: {item: i}};
});
```

PATCH each service with URL:

```
{{ 'http://mock-api:3000/services/' + encodeURIComponent($json.id) }}
```

Body:

```
{{ ({
  score: $json.score, readinessStatus: $json.readinessStatus,
  assessedAt: $json.assessedAt
}) }}
```

After the PATCH, IF `{{ Boolean($('Score readiness').item.json.finding) }}` is true, return that finding from a **Run Once for Each Item** Code node, then call LAB 00:

```
return {json: $('Score readiness').item.json.finding};
```

Step 3 - Verify stages and next-step tasks

After reset, expect stages 3, 2 and 1 for TaskFlow API, Auth0 and Stripe respectively. Expect three readiness tasks describing a next step or blocker. Those are exercise values, not current claims about the vendors.

To practice the original Switch pattern, place a Switch after Score readiness and group 0-1 as discovery/roadmap work, 2-3 as testing work, 4 as change planning, and 5 as monitor. Keep the same save-and-task behavior on the relevant branches. The Switch is an optional routing exercise, not required for correct scoring.

Step 4 - Test the evidence sequence

Load svc-api in the website. Add a value such as `LAB-HYBRID-TEST-001` for `evidence.hybridTest`, preserving its other evidence fields. Save and rerun. The score should rise to 4. Add a production reference while simulated remains true: it stays at 4. Clear the inventory reference and rerun: it falls to 0 rather than skipping missing evidence.

Done when: the stage and scope are visible, missing earlier evidence blocks advancement, a blocker remains explicit, and tasks describe the next action rather than merely saying do PQC.

15. Workflow 5 - Vendor PQC Monitoring

Goal: distinguish ready/testing/unknown/blocked states, detect stale reviews and create work for changed evidence.

Step 1 - Read the dates correctly

`lastUpdated` is the source's change date. `lastCheckedAt` is your successful retrieval time. `lastReviewedAt` is the last human review. `nextReview` is the review deadline. Fetching data today must not pretend that a person reviewed it today.

The six vendor records use `sourceType=simulation`. They do not represent verified current statuses of Auth0, AWS, Stripe, SendGrid, Intercom or Sentry.

Step 2 - Build the monitor

Create `LAB 05 - Vendor PQC Monitor`: Manual Trigger -> GET `/vendors` -> Split collection -> Code `Review vendors`, **Run Once for All Items**:

Copyable file: `code/05-vendor-review.js`

```
// Run Once for All Items. All source records are mock vendor data.
const now=Date.now(), allowed=['ready','testing','unknown','blocked'];
return $input.all().map((item,i)=>{
 const v={...item.json,pqcStatus:String(item.json.pqcStatus||'').trim().toLowerCase()},
due=Date.parse(v.nextReview), reasons=[];
 const changed=v.lastObservedStatus!==null&&v.lastObservedStatus!
==undefined&&v.lastObservedStatus!==v.pqcStatus;
 if(!allowed.includes(v.pqcStatus))reasons.push('Unrecognized status');
 if(['unknown','blocked'].includes(v.pqcStatus))reasons.push('Status requires human review');
 if(!Number.isFinite(due)||due<=now)reasons.push('Evidence review is due or date invalid');
 if(changed)reasons.push('Mock status changed: '+v.lastObservedStatus+' to '+v.pqcStatus);
 const finding=reasons.length?{dedupeKey:'vendor-review:'+v.id,ruleId:'vendor-review',
  assetId:v.id,title:'Review vendor evidence: '+v.vendor,priority:'MEDIUM',
  owner:v.owner||'Unassigned',reason:reasons.join('; '),
  observedAt:new Date(now).toISOString()}:null;
 return {json:{...v,changed,finding,lastCheckedAt:new Date(now).toISOString(),
  lastObservedStatus:v.pqcStatus},pairedItem:{item:i}};
});
```

PATCH each `/vendors/ID` with the output's lastCheckedAt and lastObservedStatus:

```
{{ ({
  lastCheckedAt: $json.lastCheckedAt,
  lastObservedStatus: $json.lastObservedStatus
}) }}
```

After the PATCH, test `{{ Boolean($('Review vendors').item.json.finding) }}`. On true, return `$('Review vendors').item.json.finding` as the json value from a **Run Once for Each Item** Code node and call LAB 00.

Step 3 - Test the states

After reset, expect review tasks for Stripe, SendGrid and Intercom. An unchanged second run should not multiply tasks or repeat the initial notifications.

Change Auth0's mock pqcStatus to blocked in the website, save and rerun. A review task should appear. Change it to ready and rerun: the change is still something to review; it does not automatically prove your integration has migrated or close every related task.

The original Switch exercise can route the normalized pqcStatus into ready, testing, unknown and blocked branches, with a fallback for unexpected values. Unknown requires research; blocked requires dependency planning; testing is not production readiness; ready still requires product/scope evidence.

Step 4 - Record an actual lab review separately

After inspecting a record, use a manual PATCH to set lastReviewedAt to a current timestamp, nextReview to a future deadline, and evidenceRef to your lab review identifier. Do not let a scheduled fetch set those human-review fields automatically.

Once tested, a weekly schedule is adequate for this practice monitor. A real vendor monitor would use official sources and record the exact product, protocol, region and availability stage rather than promoting a marketing claim directly to a migration score.

Done when: the initial risk scenarios appear, changes are detected, unexpected states are visible and check dates do not masquerade as review evidence.

16. Workflow 6 - Secret Rotation Simulator

Goal: practice replacing a credential version using metadata only. The consumer is the application that uses the credential. There is no real Stripe or SendGrid key in this exercise.

Step 1 - Detect an overdue record

Create `LAB 06A - Detect Secret Rotation`: Manual Trigger -> GET `/secrets` -> Split collection -> Code `Find overdue secrets` -> LAB 00. Use **Run Once for All Items**:

Copyable file: `code/06-secret-findings.js`

```
// Run Once for All Items. The records contain metadata, never API key values.
const now=Date.now(),out=[];
$input.all().forEach((item,i)=>{
 const s=item.json,last=Date.parse(s.lastRotated),policy=Number(s.rotationPolicyDays);
 const bad=!Number.isFinite(last)||last>now||(!Number.isFinite(policy)||!(policy>0)), age=(now-
last)/86400000;
 if(!bad&&age<=policy)return;
 const ruleId=bad?'secret-data-quality':'secret-rotation';
 out.push({json:{dedupeKey:ruleId+':'+s.id,ruleId,assetId:s.id,
  title:'Review secret metadata: '+s.name,priority:'HIGH',owner:s.owner||'Unassigned',
  reason:bad?'Invalid rotation timestamp or policy':'Secret is overdue under the lab rotation
policy',
  requestId:'rotate-'+s.id,observedAt:new Date(now).toISOString(),
  evidence:{ageDays:bad?null:age,rotationPolicyDays:policy}},pairedItem:{item:i}});
});
return out;
```

Expect one HIGH task for secret-001. Secret-002 is recently rotated and should not trigger. Older than policy means overdue under that policy, not necessarily expired or revoked by the vendor.

Step 2 - Create and approve the request

After LAB 00, require ruleId to equal secret-rotation. GET `/rotation-requests` with `id_eq=rotate-secret-001`. Only when no record exists, POST:

Copyable file: `examples/secret-rotation-request.json`

```
{
  "id": "rotate-secret-001",
  "kind": "secret",
  "assetId": "secret-001",
  "newVersion": "v2",
  "status": "pending-approval",
  "approvedBy": null,
  "approvedAt": null,
  "simulated": true
}
```

Inspect it in the website. Use a separate manual approval workflow to PATCH its status to approved, approvedBy to Lab Operator and approvedAt to `new Date().toISOString()`, as in Chapter 13. Leave an existing request's progress intact on repeated detector runs.

Step 3 - Gate and stage

Create `LAB 06B - Approved Secret Cutover`. GET the request and require approved status, kind secret, assetId secret-001, simulated true, a nonempty approver and a valid approval timestamp. Stop on false. GET `/secrets/secret-001` and require currentVersion and consumerVersion both equal v1.

PATCH the request to running. PATCH the secret with:

```
{
  "candidateVersion":"v2",
  "previousVersion":"v1",
  "oldVersionEnabled":true
}
```

Read it again. The old version must still be enabled. A candidate is prepared, not yet a completed replacement.

Step 4 - Switch the consumer and check it

PATCH the secret with `{"consumerVersion":"v2","consumerHealth":"pass"}`. This records a simulated test, not a real payment API call. GET it in a fresh node and require candidateVersion v2, consumerVersion v2 and consumerHealth pass.

On true, finalize:

```
{{ ({
  currentVersion: 'v2', candidateVersion: null,
  oldVersionEnabled: false,
  lastRotated: new Date().toISOString()
}) }}
```

The lastRotated timestamp is updated after the new consumer passes, not when the request is first submitted.

Step 5 - Verify, then close

GET `/secrets/secret-001` again and check currentVersion v2, consumerVersion v2, consumerHealth pass and oldVersionEnabled false. Confirm lastRotated belongs to this run. PATCH the request to completed.

Find the task by `dedupeKey_eq=secret-rotation:secret-001`, require exactly one row, then PATCH its ID to closed with verifiedAt and a meaningful closureReason. Read the task back. Rerun LAB 06A; it should not raise a new age finding for the recently rotated record.

Step 6 - Test the failure branch

Reset and repeat, but set consumerHealth fail at Step 4. The old version must remain enabled. For a non-compromise rollback, restore consumerVersion to v1, clear candidateVersion, leave currentVersion v1 and oldVersionEnabled true, and mark the request rolled-back. Keep its task open.

Real vendors differ in overlapping-key support, authorization and revocation behavior. This simulation does not establish those capabilities at any named vendor.

Done when: approval is required, the consumer switches before old-version retirement, failure preserves recovery, and the rotation timestamp reflects verified completion.

17. Workflow 7 - GitHub / CI Crypto Gate

Goal: receive a mock pull-request event, inspect supplied source content, store a commit-specific decision and create contextual review tasks. No GitHub repository is connected and no real merge is blocked.

Step 1 - Configure the receiver

Create `LAB 07 - CI Crypto Review`. Add Webhook: method POST, path `tf-ci-crypto`. Choose Header Auth and create a credential named `TaskFlow Lab Webhooks`. Header name: `X-Lab-Token`; value: a practice-only string you choose. Do not publish that value in screenshots or source files.

Set Respond to **Using Respond to Webhook Node**. For the website sender, set Allowed Origins as explained in Chapter 9.3. Use the exact Test URL n8n displays. The Test URL works while listening; the Production URL works when the workflow is published/active. Here production is n8n's URL label, not permission to use real production data. [R15]

Step 2 - Send the provided event

The file `events/ci-risky.json` includes repository, pullRequest, eventId, headSha, and files with both path and content. Filenames alone cannot support a content scan. The content strings are inspected, never executed.

Click Listen for test event and use the website sender or Appendix B's curl command. Confirm that the Webhook node receives the object under `$json.body`.

Step 3 - Validate and branch

Add Code `Validate CI event`, **Run Once for All Items**:

Copyable file: `code/07-validate-ci.js`

```
// Run Once for All Items; immediately after the n8n Webhook node.
const p=$input.first().json.body;
const valid=!!p&&p.repository==='taskflow'&&Number.isInteger(p.pullRequest)&&
 p.pullRequest>0&&typeof p.eventId==='string'&&/^LAB-CI-[a-zA-Z0-9-]+$/.test(p.eventId)&&
 typeof p.headSha==='string'&&/^[a-f0-9]{40}$/.test(p.headSha)&&
 Array.isArray(p.files)&&p.files.length>0&&p.files.length<=20&&p.files.every(f=>
  f&&typeof f.path==='string'&&f.path.length<=240&&typeof
f.content==='string'&&f.content.length<=50000);
return [{json:valid?{...p,valid:true}:{valid:false,error:'Invalid lab CI payload'}}];
```

Add an IF on valid equals true. The false branch goes to Respond to Webhook with status 400 and JSON body expression `{{ ({error: $json.error}) }}`. It must not create a task.

On true, add Code `Scan CI content`, **Run Once for All Items**:

Copyable file: `code/07-scan-ci.js`

```
// Run Once for All Items. Only run on the valid=true branch.
const p=$input.first().json;
// This is a policy-controlled local simulation, not payload-controlled policy.
const enforce=false;
const pattern=/\b(RSA(?:-\d+)?|RS256|PS256|ES256|ECDSA|ECDHE|Ed25519|X25519)\b/gi;
const findings=[];
for(const file of p.files){
 const matches=[...new Set((file.content.match(pattern)||[]).map(s=>s.toUpperCase()))];
 for(const algorithm of matches) findings.push({
  dedupeKey:'ci:'+p.repository+':'+p.headSha+':'+file.path+':'+algorithm,
  ruleId:'ci-crypto-review',assetId:p.repository+':'+file.path,
  title:'Review '+algorithm+' in '+file.path,priority:'MEDIUM',owner:'Security',
  reason:'Pattern match in supplied mock source text; confirm actual use and approved
exceptions',
  observedAt:new Date().toISOString(),evidence:{headSha:p.headSha,algorithm,path:file.path}
 });
}
return [{json:{...p,findings,decision:findings.length?(enforce?'simulated-block':'review-
required'):'no-pattern-found',
 checkId:'ci-'+p.pullRequest+'-'+p.headSha,mode:enforce?'enforce':'review'}}];
```

The policy-controlled enforce flag starts false. A finding therefore means review-required, not an automatic claim that all classical cryptography is prohibited. A real GitHub webhook requires signature validation over the actual body, not this practice header. [R20]

Step 4 - Save the decision for this commit

GET `/ci-checks` with `id_eq={{ $json.checkId }}`. If its response body is empty, POST `/ci-checks`. If it contains one row, PATCH that row's ID. Reject more than one match. For both writes use:

```
{{ ({
  id: $('Scan CI content').first().json.checkId,
  repository: $('Scan CI content').first().json.repository,
  pullRequest: $('Scan CI content').first().json.pullRequest,
  headSha: $('Scan CI content').first().json.headSha,
  decision: $('Scan CI content').first().json.decision,
  mode: $('Scan CI content').first().json.mode,
  checkedAt: new Date().toISOString(), simulated: true
}) }}
```

Read the saved row and check headSha. A pull request can later contain different code, so the commit identifier matters.

Step 5 - Keep both the findings and clean paths

After the saved-check read, restore the summary in a **Run Once for All Items** Code node:

```
return [{json: $('Scan CI content').first().json}];
```

IF findings.length is greater than zero, split the findings using another All Items Code node:

```
return $input.first().json.findings.map(finding => ({json: finding}));
```

Call LAB 00, then return one copy of the Scan CI content summary using the restore code above. On the no-findings branch, keep the existing summary. Connect both paths to Respond to Webhook, status 200, JSON body expression:

```
{{ ({
  decision: $json.decision,
  headSha: $json.headSha,
  findingCount: $json.findings.length,
  simulated: true
}) }}
```

Without that clean branch, splitting an empty list would end the path before a response. HTTP 200 means the event was processed, not that all security checks passed.

Step 6 - Test risky, clean, invalid and repeated events

ci-risky should produce two findings and one CI-check record. Send it again: commit-scoped tasks should not duplicate. ci-clean should produce no-pattern-found and zero new review tasks. Remove the files content from a payload: expect 400 and no mutation.

String matching can match comments and miss indirect dependencies. Real enforcement needs scoped policy, exceptions and an authenticated required status check or ruleset. This lab stores a decision only.

Done when: invalid events are rejected, content is inspected, clean events receive a response, and repeated deliveries do not multiply commit findings.

18. Workflow 8 - Crypto Incident Response

Goal: accept a simulated alert, map scope, assign work, approve a specific action and verify the changed condition. This is a certificate-replacement drill, not a fabricated real CVE or a claim that RSA has suddenly been broken.

Step 1 - Create incident intake

Create `LAB 08A - Incident Intake`. Add a POST Webhook with path `tf-security-alert`, the same practice Header Auth credential, and Respond through Respond to Webhook. Add Code `Validate incident`, **Run Once for All Items**:

Copyable file: `code/08-validate-incident.js`

```
// Run Once for All Items; immediately after the authenticated Webhook node.
const p=$input.first().json.body;
const valid=!!p&&typeof p.eventId==='string'&&/^LAB-INC-[a-zA-Z0-9-]+$/.test(p.eventId)&&
 p.simulated===true&&['high','critical'].includes(p.severity)&&
 Array.isArray(p.affectedAssetIds)&&p.affectedAssetIds.length>0&&p.affectedAssetIds.length<=20&&
 p.affectedAssetIds.every(id=>typeof id==='string'&&/^[a-z0-9-]+$/.test(id))&&
 typeof p.reason==='string'&&p.reason.length>0&&p.reason.length<=1000;
return [{json:valid?{...p,valid:true}:{valid:false,error:'Invalid simulated incident payload'}}];
```

IF valid is false, respond 400 with the error and stop. The event must identify itself as simulated, provide a LAB-INC ID and supply bounded affectedAssetIds. The algorithm name alone never authorizes a destructive action.

Step 2 - Map scope and create tasks

On true, GET `/crypto-assets`. Add Code `Build incident tasks`, **Run Once for All Items**. Keep the earlier validator named exactly `Validate incident`:

Copyable file: `code/08-incident-tasks.js`

```
// Run Once for All Items, after Get affected inventory (full HTTP response).
// The earlier validation node MUST be named Validate incident.
const p=$('Validate incident').first().json;
const assets=$input.first().json.body;
if(!Array.isArray(assets))throw new Error('Inventory response is not an array');
return [...new Set(p.affectedAssetIds)].map(id=>{
 const a=assets.find(x=>x.id===id);
 return {json:{dedupeKey:'incident:'+p.eventId+':'+id,ruleId:'incident',assetId:id,
  title:'Investigate '+p.eventId+': '+(a?a.service:id),priority:p.severity.toUpperCase(),
  owner:a&&a.owner?a.owner:'Security',reason:a?p.reason:'Unknown affected asset; validate scope
before remediation',
  observedAt:new Date().toISOString(),eventId:p.eventId,
  evidence:{simulated:true,inventoryMatched:!!a,advisoryId:p.advisoryId||null}}};
});
```

Call LAB 00 for those items. Aggregate to one result with an All Items Code node:

```
return [{json: {
  eventId: $('Validate incident').first().json.eventId,
  status: 'triaged',
  taskCount: $input.all().length,
  simulated: true
}}];
```

Respond HTTP 200 with `{{ $json }}`. Triage means work has been created, not resolved. An unknown affected asset produces a scope-validation task rather than disappearing from the blast-radius assessment.

Step 3 - Send and inspect the drill

Use `events/incident.json` through the website or Appendix B. It names cert-001 under a stable LAB-INC-001 identifier. Send it twice before remediation. Expect one task for that event-and-asset pair.

In the website, read the certificate and the incident task. Record the current fingerprint and expiry before approving a change. The next steps store that before-state in the task so verification has something independent to compare against.

Step 4 - Capture before-state and approve

Create a manual approval workflow. GET `/tasks` with `dedupeKey_eq=incident:LAB-INC-001:cert-001`. Use Chapter 10.4 to require one record and name that Code node `Selected incident task`. GET `/certificates/cert-001`, naming that node `Read original certificate`.

PATCH the task ID from `Selected incident task` with:

```
{{ ({
  approvedBy: 'Lab Operator',
  approvedAt: new Date().toISOString(),
  plannedAction: 'replace-mock-certificate',
  evidence: {
    ...$('Selected incident task').first().json.evidence,
    beforeFingerprint: $json.body.fingerprint,
    beforeExpiresAt: $json.body.expiresAt
  }
}) }}
```

The PATCH URL is `http://mock-api:3000/tasks/` plus `$('Selected incident task').first().json.id`. Execute this approval only after inspecting the intended target. Read the task back to confirm its stored evidence.

Step 5 - Remediate the approved target

Create `LAB 08B - Approved Certificate Remediation`. GET the selected incident task by its stored ID, and name the node `Get incident task`. Continue only when it is open, has assetId cert-001, has a nonempty approvedBy and valid approvedAt, and plannedAction equals replace-mock-certificate. Otherwise stop.

PATCH `/certificates/cert-001` with:

```
{{ ({
  expiresAt: new Date(Date.now() + 90*86400000).toISOString(),
  fingerprint: 'LAB-CERT-REPLACED-001',
  replacementRequestedAt: new Date().toISOString()
}) }}
```

Do not close the task from the PATCH acknowledgement. Add a separate GET `/certificates/cert-001`.

Step 6 - Verify the technical condition

Continue only when the fresh GET has id cert-001, a nonempty fingerprint different from the nonempty saved beforeFingerprint, and a valid expiry more than 30 days ahead. One boolean expression is:

```
{{ $json.body.id === 'cert-001' &&
   Boolean($json.body.fingerprint) &&
   Boolean($('Get incident task').first().json.body.evidence.beforeFingerprint) &&
   $json.body.fingerprint !==
     $('Get incident task').first().json.body.evidence.beforeFingerprint &&
   Number.isFinite(Date.parse($json.body.expiresAt)) &&
   Date.parse($json.body.expiresAt) > Date.now() + 30*86400000 }}
```

On true, PATCH the incident task ID from Get incident task with:

```
{{ ({
  status: 'closed', verifiedAt: new Date().toISOString(),
  closureReason: 'Mock replacement verified by fresh GET',
  evidence: {
    ...$('Get incident task').first().json.body.evidence,
    afterFingerprint: $json.body.fingerprint,
    afterExpiresAt: $json.body.expiresAt
  }
}) }}
```

GET the task again and confirm status closed. On false, keep it open and stop further automatic action. A valid new expiry can also support closing the separate expiry finding, but not the PQC-assessment finding.

Step 7 - Exercise failure and replay

No approval must mean no write. An unchanged fingerprint or expiry still inside the warning window must mean no closure. A later replay of the same event can reopen a closed task through this lab's generic helper; a production system needs an event ledger and explicit replay policy in addition to task deduplication.

Real verification would inspect the certificate actually served, its trust chain and hostname, client connectivity and application health. Updating inventory metadata alone would not establish production remediation.

Done when: scope is explicit, malformed events cannot act, a named change is approved, fresh evidence proves the change, and only related findings close.

Part IV - Reliability and everyday operation

19. Make the lab reliable without overstating it

19.1 Keep inventory, policy, evidence and action separate

Layer	Question	Example
Inventory	What exists?	Certificate cert-001 belongs to Platform.
Policy	What is required?	Warn below 30 days; review age above 90 days.
Evidence	What was observed?	UTC expiry, served fingerprint, review reference.
Action	What happens next?	Create work, obtain approval, change, verify, close.

The source's closed-loop design depends on this separation. A mock success response is not independent evidence that a real system changed. An empty scan result does not authorize closing old tasks; it could mean a wrong filter, unavailable data or an incomplete scan.

19.2 Deduplication is not complete transaction safety

LAB 00 prevents duplicate findings during sequential runs by looking up a stable dedupeKey. Two overlapping executions can still both see no row and both create one. Keep manual exercises sequential. A real datastore needs a unique constraint, atomic upsert or another concurrency control.

Task creation and notification creation are separate writes. A crash between them can create a task without its initial notification. The lab is not an exactly-once delivery system. A production design needs a durable delivery record/outbox, retries and reconciliation.

19.3 Error handling and retries

Failure	What to do
GET timeout or transient 5xx	Use a small bounded retry, then fail visibly.
HTTP 401/403	Check credentials and permissions; do not blindly retry.
HTTP 429	Respect Retry-After where supplied and reduce frequency.
Invalid JSON/date/required field	Route to data-quality work; do not infer healthy state.
POST timeout	Read current state first; the write may have succeeded.
Approval or verification failure	Stop the action branch; keep the finding open.
Missing API after restart	Restore or initialize records before continuing.

For a read-only HTTP node, Settings -> Retry on Fail can use a small limit, for example three attempts and 1000 ms between attempts. Do not apply automatic blind retries to a multi-step rotation or POST. Avoid Continue On Fail on the very check that protects a destructive action.

19.4 Create and test a shared error workflow

Create `LAB ERR - Execution Failure` with Error Trigger. Add Edit Fields to retain a short message, failed node and workflow name from the trigger's output. Then POST `/notifications` with priority HIGH, owner Lab Operator, the message, and a current createdAt. Do not copy entire sensitive payloads into error records.

In each main workflow's Settings, select LAB ERR as its error workflow. Test it using a temporary scheduled workflow calling GET `/simulate-error`, with Never Error off. Publish that temporary workflow, observe an automatic failure and the resulting notification, then unpublish it. Error Trigger does not fire for a manual parent execution. [R21]

When Mockoon is unavailable, a notification POST to Mockoon also fails. Inspect n8n Executions and Docker logs. A real deployment needs an independent alert destination.

19.5 Optional SSH-key and RDS exercises

The ssh-001 inventory row represents an ordinary Ed25519 key. Calculate its age from createdAt and inactivity from lastUsedAt separately. A plain SSH key does not inherently have a certificate-style expiry. Create an ssh-key-review finding when the lab age policy is exceeded. Obtain owner confirmation and replacement-access checks before a simulated `accessStatus=disabled` update. Read it back before closing work.

The tls-001 row represents a simulated RDS connection observation. Require an allowed tlsVersion, certificateValidated true, hostnameValidated true and a recent observedAt. Missing or false values require investigation. PATCHing these flags only simulates a corrected measurement; it does not validate an actual database connection. Certificate verification and encrypted transport are different assurances. [R22]

19.6 Safe progression toward real systems

First replace prepared inventory with authorized, read-only non-production evidence. Next connect a real task platform while keeping cryptographic changes manual. Then test provider-specific actions in a sandbox with recovery and monitoring. Production automation requires explicit authorization, least privilege, secure credentials and actual endpoint/consumer verification.

Do not publish the unauthenticated Mockoon API, give it real credentials, or forward your router's ports to this lab. Remote providers cannot send webhooks to your Mac's localhost. Public webhooks require a separately designed authenticated HTTPS deployment; they are outside this local exercise.

20. Daily use, backup, restore and updates

20.1 Start a practice session

Open Docker Desktop and wait for the engine. In Terminal:

```
cd "$HOME/Documents/TaskFlow-Local-Lab"
docker compose up -d
docker compose ps
```

Open n8n at `http://localhost:5678` and the website at `http://localhost:3000`. If Mockoon restarted, restore a saved snapshot or initialize fresh records. Inspect counts before running a detector. Keep published schedules off until the intended mock data is present.

You do not need Mockoon Desktop open merely to run the Docker mock server. Open Desktop when editing its configuration or practicing its GUI.

20.2 Stop without deleting your work

Finish or cancel active runs, unpublish schedules you do not want to resume, save workflow edits, export a fresh mock snapshot, then run:

```
docker compose stop
```

This stops the containers without removing them or their volume. The next mock server start still loses its live in-memory state, so the snapshot matters. Closing Safari alone does not stop the containers. Quitting Docker Desktop stops its engine. [R7, R23]

20.3 Commands to recognize

Command	Meaning
`docker compose ps`	Show this project's service state.
`docker compose logs --tail=60 n8n`	Show recent n8n logs.
`docker compose logs --tail=60 mock-api`	Show recent mock server logs.
`docker compose restart n8n`	Restart n8n with its existing configuration.
`docker compose restart mock-api`	Restart Mockoon and reset its live buckets.
`docker compose up -d`	Create/start/reconcile services from the configuration.
`docker compose stop`	Stop without removing containers.
`docker compose down`	Remove project containers/network; named volumes normally remain.
`docker compose down -v`	Also remove named volumes; destructive for n8n data. Do not use routinely.

20.4 Export workflow definitions

In each n8n workflow, use its menu's download/export option to save JSON into `workflow-exports`. This preserves the workflow definition. It is not a complete backup of credentials, encryption keys or execution history. Imported cloud workflows need local credentials reconfigured and their old URLs checked.

Keep the mock environment file, website files, seed data and snapshots too. They are different artifacts from n8n workflow JSON. Do not put private backups into a portfolio repository.

20.5 Make a full n8n data backup

This sequence assumes this guide's default SQLite setup and generated encryption configuration. Stop n8n before copying its database so files are not changing during the copy. The commands deliberately leave the original volume intact. [R7, R24]

Step 1. Finish runs and export the mock snapshot. In the project folder, create a uniquely named backup folder and record the running image:

```
BACKUP="$PWD/backups/n8n-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$BACKUP/data"
CONTAINER_ID="$(docker compose ps -q n8n)"
docker inspect "$CONTAINER_ID" --format '{{.Image}}' > "$BACKUP/image-id.txt"
docker compose exec -T n8n n8n --version > "$BACKUP/n8n-version.txt"
cp compose.yaml "$BACKUP/compose.yaml"
```

Step 2. Stop only n8n, then copy its complete data directory:

```
docker compose stop n8n
docker compose cp n8n:/home/node/.n8n/. "$BACKUP/data"
ls -la "$BACKUP/data"
```

Step 3. Confirm the default database and config were copied:

```
test -f "$BACKUP/data/database.sqlite" && echo "Database present"
test -f "$BACKUP/data/config" && echo "Config present"
```

Expect both messages for this default setup. Missing files mean investigate before relying on the backup. Protect the whole backup: its database and configuration can contain credential information and the key needed to decrypt it. Do not paste them into chat or publish them.

Step 4. Resume:

```
docker compose start n8n
```

A copy existing on disk is not proof that recovery works. Keep another protected copy off the Mac and practice the isolated restore below before important use.

20.6 Optional restore drill into a NEW volume

This is a recovery exercise, not a fix for a missing port. It preserves the original volume. Use only a backup from this local lab. Restored workflows may include schedules, so keep them unpublished for the drill.

Step 1. Set BACKUP to the real saved directory, replacing the timestamp below. Confirm the two files and recorded image exist:

```
BACKUP="$PWD/backups/n8n-REPLACE_WITH_YOUR_TIMESTAMP"
ls "$BACKUP/data/database.sqlite" "$BACKUP/data/config"
IMAGE_ID="$(cat "$BACKUP/image-id.txt")"
docker image inspect "$IMAGE_ID"
```

Stop here if the path or image is missing. Recover the matching image version before restoring; do not silently use a newer release against an old database.

Step 2. Create a new uniquely named volume and copy into it with a disposable helper:

```
RESTORE_VOLUME="taskflow_restore_$(date +%Y%m%d-%H%M%S)"
docker volume create "$RESTORE_VOLUME"
docker run --rm --user 0 \
  -v "$RESTORE_VOLUME":/restore \
  -v "$BACKUP/data":/backup:ro \
  alpine:3 sh -c 'cp -a /backup/. /restore/ && chown -R 1000:1000 /restore'
```

Alpine is a small Linux helper image used here only to copy files and set ownership for n8n's default node user. It does not replace Mockoon or n8n. Docker may need to download it. The source backup is mounted read-only; the original n8n volume is not touched.

Step 3. Make a temporary Compose override identifying the restored volume and recorded image:

```
printf 'services:\n  n8n:\n    image: "%s"\n    pull_policy: never\nvolumes:\n  n8n_data:\n
name: "%s"\n' \
  "$IMAGE_ID" "$RESTORE_VOLUME" > compose.restore.yaml
```

Inspect that file with `cat compose.restore.yaml`. Then stop the main n8n and start its service using the override:

```
docker compose stop n8n
docker compose -f compose.yaml -f compose.restore.yaml up -d n8n
```

Step 4. Open Safari, sign in and confirm that the storage-check workflow and expected lab workflows exist. Test a harmless read. If credentials were used, confirm they are readable through n8n without exposing their values.

Step 5. To return to the untouched original volume, stop the restored service and start with the base file only:

```
docker compose -f compose.yaml -f compose.restore.yaml stop n8n
docker compose -f compose.yaml up -d n8n
```

Keep the restore volume until you have verified the result; cleanup is a separate deliberate action. The override is used only when explicitly named with `-f`. Do not leave uncertainty about which volume is currently active.

20.7 Update carefully, not automatically

Record `docker compose exec n8n n8n --version` and the mock server's version/startup log. After successful testing, replace the `latest` tag in compose.yaml with the exact release tag you used. A fixed tag makes later changes intentional. Keep the matching Mockoon environment format and CLI compatible.

Before updating, save workflows, make a full n8n backup, export the mock snapshot, and read the publisher's release notes for breaking changes. Choose the desired image tag, edit compose.yaml, run `docker compose pull`, then `docker compose up -d`. Re-run the health, storage and read/write/read-back tests. Do not simply point an older image at a database already migrated by a newer version; recovery may require the pre-update backup and matching old image.

Appendices

Appendix A - Mock payloads and endpoint translation

A1. The complete fixture files

The ZIP contains every collection under `examples` and the combined reset source at `mockoon/seed-data.json`. The `@days:N` notation is a reset helper marker, not a valid API timestamp. The website converts it before writing live records. Do not POST unconverted markers directly and expect the date checks to accept them.

Representative certificate:

```
{
  "id":"cert-001",
  "name":"taskflow-api",
  "service":"TaskFlow API",
  "owner":"Platform",
  "algorithm":"RSA-2048",
  "expiresAt":"@days:6",
  "fingerprint":"LAB-CERT-OLD-001",
  "environment":"lab"
}
```

Representative service:

```
{
  "id":"svc-api",
  "service":"TaskFlow API",
  "scope":"TLS key establishment",
  "crypto":["ECDHE"],
  "vendorControlled":false,
  "evidence":{
    "inventory":"LAB-INV-001",
    "roadmap":"LAB-PLAN-001",
    "testEnvironment":"LAB-ENV-001",
    "hybridTest":null,
    "production":null
  },
  "blockers":[],
  "simulated":true
}
```

Representative secret metadata:

```
{
  "id":"secret-001",
  "name":"stripe-api-key",
  "owner":"Payments",
  "lastRotated":"@days:-100",
  "rotationPolicyDays":90,
  "currentVersion":"v1",
  "candidateVersion":null,
  "consumerVersion":"v1",
  "oldVersionEnabled":true,
  "consumerHealth":"pass",
  "environment":"lab"
}
```

No actual secret value belongs in this object.

A2. The two event payloads

Risky CI event:

Copyable file: `events/ci-risky.json`

```
{
  "eventId": "LAB-CI-001",
  "repository": "taskflow",
  "pullRequest": 241,
  "headSha": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "files": [
    {
      "path": "src/auth/jwt.js",
      "content": "const signingAlgorithm = 'RS256';"
    },
    {
      "path": "src/security/crypto.js",
      "content": "const transportGroup = 'ECDHE';"
    }
  ]
}
```

Certificate incident drill:

Copyable file: `events/incident.json`

```
{
  "eventId": "LAB-INC-001",
  "simulated": true,
  "advisoryId": "LAB-ONLY-001",
  "severity": "critical",
  "algorithm": "RSA-2048",
  "affectedServices": [
    "TaskFlow API"
  ],
  "affectedAssetIds": [
    "cert-001"
  ],
  "reason": "Simulated urgent certificate-replacement exercise. This is not a claim of a real RSA
vulnerability."
}
```

A3. What happened to the first guide's action routes?

First guide sketch	Concrete operation in this edition
`POST /tasks` returns a fixed acknowledgement	CRUD POST stores a task, then a separate GET verifies it.
`POST /rotation-request`	POST `/rotation-requests` creates a pending approval record.
`POST /jwt/rotate`	POST a replacement key-metadata row, then controlled PATCH operations.
`POST /secrets/rotate`	PATCH candidate, consumer and final version state after approval.
`PATCH /.../:id` returns simulated success	CRUD PATCH updates the row for read-back during the server session.
`/webhook/github` or `/security-alert` simulation	Local n8n receives `/tf-ci-crypto` or `/tf-security-alert` webhook paths.

The lifecycle concepts are retained. Fixed acknowledgements are replaced with visible state changes; no unsupported real key-rotation capability is implied. The old invented CVE-looking string is replaced by a clearly marked LAB incident.

Appendix B - Terminal tests

Run these from Terminal on the Mac, not from inside an n8n Code node. Start the relevant servers first. These commands intentionally use localhost because Terminal is on the Mac. Inside n8n, use mock-api for the mock server.

B1. Check the API and create one disposable task

```
curl http://localhost:3000/health
curl http://localhost:3000/certificates
curl -X POST http://localhost:3000/tasks \
  -H 'Content-Type: application/json' \
  --data-binary @examples/task-create.json
```

Read the returned task ID, then GET `/tasks/THAT_ID` using that actual ID. Reset the website after this smoke test before checking baseline counts.

B2. Send a CI test event

In n8n, click Listen for test event. Replace YOUR_LAB_TOKEN below with the same practice header value saved in the Webhook credential:

```
curl -X POST http://localhost:5678/webhook-test/tf-ci-crypto \
  -H 'Content-Type: application/json' \
  -H 'X-Lab-Token: YOUR_LAB_TOKEN' \
  --data-binary @events/ci-risky.json
```

Use `@events/ci-clean.json` to test the clean path. The `@` tells curl to read the JSON from the file. Re-arm the test listener before another development request.

B3. Send the incident drill

```
curl -X POST http://localhost:5678/webhook-test/tf-security-alert \
  -H 'Content-Type: application/json' \
  -H 'X-Lab-Token: YOUR_LAB_TOKEN' \
  --data-binary @events/incident.json
```

Use the exact URL displayed by your node if it differs. To test a published workflow, use its `/webhook/...` Production URL instead of `/webhook-test/...`. A local Production URL still cannot be called from an external provider's server.

Appendix C - Troubleshooting matrix

Symptom	What to check next
Docker command not found	Finish Desktop's CLI setup, reopen Terminal and inspect its PATH/settings.
Cannot connect to Docker daemon	Open Docker Desktop and wait for the engine; do not reinstall n8n.
No Compose configuration found	Run pwd and ls; move to the folder containing compose.yaml.
Safari cannot connect to n8n	Inspect service state, the 5678 port publication and n8n logs.
Only `5678/tcp` appears	That container lacks the required host mapping; use the supplied Compose service.
Port already allocated	Stop the other server on that port. Do not start both Desktop and CLI for TaskFlow.
n8n asks to create an owner again	Verify the active project/volume; a new empty volume is a different instance.
Mockoon rejects environment format	Open/save in a compatible Desktop release and use a compatible CLI image.

Symptom	What to check next
Website downloads instead of displaying	Root route must send the HTML file as body with the HTML Content-Type.
Website displays raw JSON	Open `/`, not `/certificates` or another data route.
Website cannot find scripts or styles	Keep the supplied folder together; verify file routes and relative file paths.
Empty collections immediately after startup	Initialize through the website or restore your snapshot.
Data disappears after restart	Expected Mockoon bucket behavior; n8n data and mock snapshots have separate lifecycles.
Browser works but n8n cannot reach mock API	Use `http://mock-api:3000`, verify both services share the Compose project.
Full-response code says body is not an array	Enable the documented response settings; confirm the route returns a list, not HTML or one row.
A linked-node expression is undefined	Check exact node names, Code execution mode and whether that branch executed.
Duplicates accumulate	Check stable dedupeKey, equality filter and overlapping runs.
Rotation has an existing candidate or running request	Inspect the partial state; do not blindly repeat POST or reset approval.
Test webhook returns 404	Click Listen for test event and use the exact current Test URL and POST method.
Webhook returns 401/403	Verify Header Auth name/value and any IP restrictions.
Website event sender reports CORS failure	Check allowed origin and n8n execution; use curl to separate browser rules from server reachability.
Clean event never receives a response	Preserve the no-findings path instead of splitting an empty list.
Error Trigger does not run during manual testing	Test an automatic parent execution with the error workflow selected.

Restricted private-IP / SSRF error

Some n8n configurations enable server-side request-forgery protection, which may block internal destinations. Only when that is the actual error, allow the specific service hostname you control rather than disabling protection broadly:

```
N8N_SSRF_ALLOWED_HOSTNAMES: mock-api
```

Place that setting under the n8n environment section, then run `docker compose up -d n8n` to apply it. Do not allow arbitrary public input to choose internal hosts. This allowance is for the guide's dedicated mock service. [R25]

The best troubleshooting order

Check whether the service is running. Check the exact URL and method. Check the HTTP status and response shape. Check your item references and conditions only after connectivity is confirmed. Before retrying a failed write, read its current state.

Appendix D - Glossary

Term	Meaning
API / endpoint	A machine-facing interface / a particular path and HTTP operation.
Authentication	Checking that a caller supplies an accepted identity/credential.
Approval gate	A condition that must pass before a change; mock editable fields are not secure authorization.
Bind mount	A Mac folder made available inside a container.
Blast radius	The systems or dependencies potentially affected by an event.
Certificate	A signed structure binding public-key information to identity information.
Container / image	An application instance / the package used to create it.
CORS	Browser rules for cross-origin reads; not an authentication mechanism.
CRUD	Create, read, update and delete operations.
Crypto agility	The ability to replace cryptographic choices without rebuilding everything.
Deduplication / idempotency	Avoiding duplicate work / repeated processing without unintended repeated effects.
Environment in Mockoon	The saved mock API configuration, not an operating-system environment variable.
Environment variable	A startup setting passed to an application, such as a timezone.
Evidence reference	A pointer to a test/review/observation; not proof by itself.
Host / localhost	The computer in the relevant context / that context's own local machine.
JSON	Structured data using objects, arrays, strings, booleans, numbers and null.
JWT / JWKS / kid	JSON Web Token / JSON Web Key Set / key identifier.
Metadata	Descriptive fields, not actual keys, credentials or certificates.
Overlap / retirement	Keeping old verification available temporarily / removing old use after checks.
Port mapping	A connection from a host-side port to a container-side port.
PQC	Post-quantum cryptography; the lab tracks migration work, not implementation assurance.
Read-back	A new GET checking saved state, separate from the write acknowledgement.
Seed data	Starting sample records for predictable exercises.
Snapshot / backup	A saved set of mock records / a protected copy of n8n's persistent data in this guide.
UTC timestamp	A timezone-explicit date-time; the examples use a trailing Z.
Volume	Persistent Docker-managed storage; not automatically an off-device backup.
Webhook	An incoming HTTP request that starts a workflow.

Appendix E - Completion and preservation checklist

E1. Acceptance results from a fresh initialization

Test	Expected result
Docker/n8n	Correct loopback port; local owner login; workflow survives n8n restart.
Mockoon basics	Fixed route works; CRUD PATCH survives GET; restart restores initial bucket.
Main website	Eight assets, five certificates; edit, save, read-back and snapshot restore work.
LAB 00	Same finding creates one task; escalation creates an additional notification.
Workflow 1	Seven review tasks and eight inventory classifications.
Workflow 2	Four findings, including the deliberately invalid date.
Workflow 3	One overdue-key finding; approved stage/cutover/overlap/retirement sequence.
Workflow 4	Scoped stages 3, 2 and 1; three next-step/blocker tasks.
Workflow 5	Three initial vendor-review tasks.
Workflow 6	One overdue-secret task; failure prevents old-version retirement.
Workflow 7	Risky event has two findings; clean event responds with zero; invalid input is rejected.
Workflow 8	One task for the provided incident; no unapproved write or unverified closure.

Counts assume the supplied fixtures, a recent reset and no unrelated edits. The same task may later be closed or reopened deliberately. Record the version, date and result of each test rather than treating a screenshot as proof of the entire system.

E2. Source preservation map

First PDF content	Where it appears in this edition
Ten automation opportunities and planning priorities	Chapter 1.1
Crypto Control Plane and closed-loop tasks	Chapters 1.2 and 1.5
Certificate/key/policy rules and PQC scoring	Chapters 1.3-1.4 and the individual workflows
Environment setup, routes and intentionally risky data	Chapters 2-9 and Appendix A
Mock TaskFlow website and event simulation	Chapter 9, supplied source files and Appendix B
All eight workflows in the original order	Chapters 11-18
Stateful upgrade, key overlap and verification	Chapters 7, 13, 16 and 18

First PDF content	Where it appears in this edition
Deduplication, errors, approvals and safe migration	Chapters 10 and 19
Inventory / policy / evidence / action separation	Chapter 19.1
Portfolio structure and safe project description	This appendix and the kit's README
Mock payloads, test commands, troubleshooting and glossary	Appendices A-D and matching kit files
Final completion checks	Appendix E1

The normal-route acknowledgements in the first edition are not reused as proof of saved changes. This edition uses CRUD buckets and read-back, clarifies date boundaries and Code-node modes, and replaces unsupported real-world implications with explicit simulations. No cloud account is required for the main exercise.

E3. Portfolio description

Describe the project as a **local security-automation lab for cryptographic lifecycle and PQC migration planning using invented data**. Include the configuration, sanitized workflow exports, mock fixtures, test notes and screenshots. Exclude backups, credentials and real customer information.

A good demonstration shows a failing condition, its task, an approval, a simulated change, failed-verification behavior and successful verified closure. Do not describe it as a certified scanner or a completed production PQC deployment.

References and implementation notes

[S1] TaskFlow_Security_Automation_Lab_Guide.pdf, original 34-page edition supplied in this conversation. The source provides the four-part strategy/lab/workflow/reliability organization, ten opportunities, eight workflows, payload concepts and closed-loop model. Docker/Mockoon procedures, the supplied kit and concrete stateful checks are implementation expansions in this edition.

The following official references support software-specific setup and behavior. Documentation checked on 9 October 2026. Interface labels and moving image tags may change; record the versions actually installed. The sample policies, fixtures, code and acceptance tests are lab designs, not vendor attestations.

[R1] n8n - Compare editions. https://docs.n8n.io/hosting/community-edition-features/ [R2] Docker - Install Docker Desktop on Mac, requirements and eligibility. https://docs.docker.com/desktop/setup/install/mac-install/ [R3] Mockoon - Desktop downloads. https://mockoon.com/download/ [R4] Docker - Port publishing and mapping. https://docs.docker.com/engine/network/port-publishing/ [R5] Docker - Networking in Compose. https://docs.docker.com/compose/how-tos/networking/ [R6] Mockoon - CRUD routes and filters. https://mockoon.com/docs/latest/api-endpoints/crud-routes/ [R7] Docker - Volume lifecycle and backups. https://docs.docker.com/engine/storage/volumes/ [R8] n8n - Security environment variables. https://docs.n8n.io/deploy/host-n8n/configure-n8n/basic-configuration/use-environment-variables/security [R9] n8n - Docker installation and persistent data. https://docs.n8n.io/deploy/host-n8n/install-options/install-with-docker [R10] Mockoon - Create your first mock API. https://mockoon.com/tutorials/getting-started/ [R11] Mockoon - Listening hostname/adapter. https://mockoon.com/docs/latest/server-configuration/listening-hostname/ [R12] Mockoon - Data bucket creation and lifecycle. https://mockoon.com/docs/latest/data-buckets/overview/ [R13] Mockoon - Opening and sharing mock API files. https://mockoon.com/docs/latest/mockoon-data-files/sharing-mock-api-files/ [R14] Mockoon - File serving and relative paths. https://mockoon.com/docs/latest/response-configuration/file-serving/ [R15] n8n - Webhook node. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/ [R16] n8n - HTTP Request and Code nodes. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/ and https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.code/ [R17] n8n - Execute Sub-workflow and its trigger. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.executeworkflow/ and https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.executeworkflowtrigger/ [R18] NIST - Post-Quantum Cryptography project. https://csrc.nist.gov/projects/post-quantum-cryptography [R19] IETF - RFC 7517, JSON Web

Key. https://www.rfc-editor.org/rfc/rfc7517.html [R20] GitHub - Validating webhook deliveries. https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries [R21] n8n - Error Trigger. https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.errortrigger/ [R22] OpenSSH ssh-keygen and PostgreSQL TLS validation. https://man.openbsd.org/ssh-keygen and https://www.postgresql.org/docs/current/libpq-ssl.html [R23] Docker - Compose lifecycle commands. https://docs.docker.com/reference/cli/docker/compose/ [R24] Docker - Copy files from a Compose service. https://docs.docker.com/reference/cli/docker/compose/cp/ [R25] n8n - SSRF protection and narrow allowances. https://docs.n8n.io/deploy/host-n8n/configure-n8n/basic-configuration/use-environment-variables/ssrf-protection [R26] Mockoon - CLI and Docker usage. https://mockoon.com/cli/

The supplied Docker configuration is for a loopback-only, single-user practice environment. Live Docker Desktop, Mockoon Desktop and n8n integration must be checked on the user's Mac. See the kit's VALIDATION.txt for the precise checks performed on the companion files; those checks are not a claim that this complete stack was installed in your accounts or on your computer.